Ollscoil Teicneolaíochta an Atlantaigh

Atlantic Technological University

Adaptive AI Programming Quiz Platform

By
Fionn McGoldrick

April 26, 2026

B.Sc. (Hons) in Software Development

Minor Dissertation

Department of Computer Science & Applied Physics,
School of Science & Computing,
Atlantic Technological University (ATU), Galway.

Contents

Abstract **2**

1 Introduction **3**
1.1 Context and Motivation . . . 3
1.2 Project Scope and Level . . . 4
1.3 Objectives . . . 5
1.4 Chapter Overview . . . 6
1.5 Project Repository . . . 6

2 Methodology **7**
2.1 Development Approach . . . 7
2.2 Planning and Requirements . . . 8
2.3 Iterative Development Cycles . . . 8
2.4 GitHub Workflow . . . 9
2.5 Continuous Integration Pipeline . . . 10
2.6 Validation and Testing . . . 11
2.7 Technology Selection Criteria . . . 12

3 Technology Review **13**
3.1 Overview . . . 13
3.2 Large Language Models and Generative AI . . . 13
3.2.1 The Emergence of Large Language Models . . . 13
3.2.2 Prompt Engineering . . . 14
3.3 Machine Learning Foundations . . . 15
3.3.1 Text Representation with TF IDF . . . 15
3.3.2 Logistic Regression . . . 15
3.3.3 Multi Label Classification . . . 16
3.3.4 Gradient Boosting for Difficulty Prediction . . . 16
3.3.5 Proxy Labels and Class Imbalance . . . 17
3.4 Datasets . . . 17
3.4.1 LeetCode Dataset for Topic Tag Classification . . . 17
3.4.2 IBM Project CodeNet . . . 18
3.5 Backend Framework: FastAPI . . . 18
3.6 Database: PostgreSQL . . . 19
3.7 Containerisation: Docker and Docker Compose . . . 19
3.8 Frontend: React and TypeScript . . . 20
3.9 Authentication: JSON Web Tokens and bcrypt . . . 20
3.10 Code Execution Engine . . . 21
3.11 Data Interchange and Continuous Integration . . . 21

4 System Design **24**
4.1 Overview . . . 24
4.2 Docker Compose Infrastructure . . . 25
4.3 Backend Architecture . . . 25
4.3.1 API Routes . . . 26
4.3.2 AI Generation Pipeline . . . 28
4.3.3 Machine Learning Inference Pipeline . . . 28
4.3.4 Difficulty Classifier Training Pipeline . . . 29
4.3.5 Topic Tag Classifier Training Pipeline . . . 30
4.3.6 Code Execution Engine . . . 31
4.3.7 Authentication and User Management . . . 31
4.4 Database Design . . . 34
4.4.1 Users Table . . . 35
4.4.2 Quiz Results Table . . . 35
4.4.3 Friendships Table . . . 35
4.5 Frontend Architecture . . . 35
4.5.1 Route Structure . . . 36
4.5.2 State Management . . . 36
4.5.3 Key Page Components . . . 36
4.6 End to End Data Flow . . . 38
4.7 Deployment Architecture . . . 39
4.8 Design Decisions and Standards Compliance . . . 39

5 System Evaluation **40**
5.1 Introduction . . . 40
5.2 Objective 3: Difficulty Classifier . . . 40
5.2.1 Dataset and Class Balance . . . 40
5.2.2 Model Comparison . . . 41
5.2.3 Final Model Results . . . 42
5.2.4 Critical Analysis: The Proxy Label Problem . . . 44
5.3 Objective 2: Topic Tag Classifier . . . 44
5.3.1 Dataset Analysis . . . 44
5.3.2 Feature Analysis . . . 46
5.3.3 Model Comparison . . . 48
5.3.4 Final Model Test Set Results . . . 52
5.3.5 Critical Analysis: Class Imbalance and Conceptual Overlap . 55
5.4 Application Level Evaluation . . . 56
5.4.1 Objective 1: AI Generation Pipeline . . . 56
5.4.2 Objective 5: Code Execution Engine . . . 57
5.4.3 Objective 6: User Features . . . 57
5.5 Evaluation Against Objectives . . . 58

6 Conclusion **60**
6.1 Summary of Context and Objectives 60
6.2 Summary of Findings 60
6.3 Unexpected Insights 61
6.4 Opportunities for Future Investigation 62
6.5 Closing Remarks 63

A Installation and Repository **70**

List of Figures

4.1 High level system architecture showing the three Docker Compose services and their communication paths. 25
4.2 Backend module dependency diagram showing call relationships between the FastAPI entry point, service modules, AI parsers, and machine learning inference wrappers. 26
4.3 Flowchart of the code execution engine, tracing the path from an incoming request through runtime selection, subprocess execution, test harness injection, and result evaluation. 32
4.4 Entity relationship diagram for the four database tables. The users table is the central entity, with one to many relationships to both quiz_results and quiz_sessions. The friendships table models a directed self referential relationship on users, with a unique constraint preventing duplicate requests and a check constraint preventing self friendship. 34
4.5 End to end data flow for a quiz generation session, showing the request path from browser through JWT authentication, AI generation via OpenAI or Ollama, and ML enrichment. MCQ questions receive a predicted difficulty label only, while coding challenges receive both difficulty and topic tags. Both models are loaded at startup as in process singletons. The session concludes with results persisted and experience points awarded. 38

5.1 Class distribution of the difficulty labels derived from acceptance rate percentile binning across the CodeNet dataset. The near equal counts across easy, medium, and hard confirm that the tercile binning strategy produced a balanced training set. 41
5.2 Validation accuracy comparison across all seven candidate difficulty classifier models after RandomizedSearchCV tuning. Model selection during cross validation was scored on F1 macro; accuracy is reported here for comparability. XGBoost achieved the highest validation accuracy at 0.605. 42

5.3 Confusion matrix for the final XGBoost difficulty classifier evaluated on the held out test set. Rows represent true labels and columns represent predicted labels. 43

5.4 LeetCode topic tag distribution before filtering (left) and after applying the 60 sample minimum threshold (right). Array dominates with 1,168 examples. Tags such as Stack (13), Binary Search (10), and Graph (8) were removed due to insufficient samples. 45

5.5 Word count distribution per topic class for the combined title and description text. Database problems are consistently longer than other classes. The wide variance within classes reflects the diverse nature of problems within each topic. 46

5.6 Top 30 terms across the full LeetCode corpus after English stop word removal. High frequency generic terms such as `example`, `input`, and `output` appear across all topic classes and provide limited discriminative value for classification. 47

5.7 Top 12 classifier weight features per topic class for the final SGD Classifier. Classes with distinctive vocabulary (Database, Linked List, Tree) show clean, semantically meaningful features. Dynamic Programming and Hash Table show more overlap with other classes, which corresponds directly to their lower F1 scores. 48

5.8 Validation set F1 macro comparison across the seven candidate topic tag classifier models. The SGD classifier achieved the highest F1 macro and was therefore selected as the final deployed model. . . . 49

5.9 Validation accuracy, F1 macro, and F1 weighted for the seven candidate topic tag classifier models. The consistent gap between accuracy and F1 macro reflects the class imbalance in the LeetCode dataset, where high accuracy can be achieved by performing well on the dominant classes alone. 50

5.10 Comparison of default parameters versus GridSearchCV tuned parameters for each model on the validation set. The most notable improvement is Multinomial NB, which gained 17 percentage points in accuracy after tuning. 51

5.11 Cross validation F1 macro score distributions (five fold) for all models with best hyperparameters. SGD Classifier and Linear SVC show the tightest distributions, indicating the most consistent generalisation across folds. 52

5.12 Per class F1 scores for all models on the validation set, with the SGD Classifier test set results shown in the final row. Array, Database, and Tree are consistently strong across all models. Dynamic Programming, Hash Table, and Two Pointers are consistently the weakest classes. 53
5.13 Row normalised confusion matrix for the SGD Classifier on the test set. Array (0.90), Database (1.00), Math (0.86), and Tree (1.00) show strong diagonal dominance. Dynamic Programming (0.40) and Two Pointers (0.43) show the highest off diagonal confusion, primarily misclassified as Array. 54
5.14 Total training time including GridSearchCV for each topic tag classifier model. All models complete training in under seven seconds, confirming that retraining is computationally trivial should the dataset be expanded in future. 56

List of Tables

5.1 Per class performance of the XGBoost difficulty classifier on the test set, derived from the confusion matrix. 43
5.2 Final SGD Classifier performance on validation and test sets. The 2.4 percentage point drop in F1 macro from validation to test set is modest and consistent with normal generalisation behaviour. 52
5.3 Per class F1 scores for the final SGD Classifier on the test set, with the primary misclassification direction noted for weaker classes. . . 55
5.4 Summary of project objectives and their evaluation outcomes. . . . 58

Abstract

This dissertation presents the design, implementation, and evaluation of a full-stack web platform that generates personalised programming quizzes and coding challenges on demand using large language model generation. The system addresses a limitation in existing technical education tools: existing platforms either offer static question banks that grow stale, or expose raw AI-generated content without structure, difficulty signals, or topic labels. The platform developed here bridges that gap by combining the generative capability of large language models with two custom-trained machine learning classifiers that automatically enrich every generated question with a predicted algorithmic topic tag and a predicted difficulty level, without requiring a second model call or any manual curation.

The topic tag classifier was trained on a corpus of LeetCode programming problems using a stochastic gradient descent estimator over a combined word-level and character-level TF-IDF feature representation. It achieved a test set accuracy of 0.805 and an F1 macro score of 0.622 across ten algorithmic topic classes. The difficulty classifier was trained on IBM Project CodeNet, a dataset of over fourteen million competitive programming submissions, using an XGBoost ensemble over engineered features derived from submission statistics and problem description text. Difficulty labels were derived from acceptance rate percentile binning in the absence of explicit human annotations, and the final model achieved a test set accuracy of 0.593 on a balanced three-class problem. Both classifiers are deployed as in-process singletons within a Python FastAPI backend, adding negligible inference latency per generated question.

The platform additionally includes a sandboxed multi-language code execution engine supporting Python, JavaScript, TypeScript, Java, and C#, a Monaco Editor coding environment, a JWT-authenticated user account system with an experience point and levelling mechanic, and a social friends network. The full system is containerised using Docker Compose and deployable with a single command. All six project objectives were met and evaluated against measurable criteria on held-out test sets.

Chapter 1

Introduction

1.1 Context and Motivation

The landscape of technical education has undergone considerable transformation in recent years, driven in large part by the proliferation of artificial intelligence tools capable of generating content on demand. Within computing and software engineering education, the ability to assess understanding and reinforce learning through practice problems, quizzes, and coding challenges has always been central to how students develop competency. However, the process of creating these materials has traditionally been a time consuming and manual endeavour, placing a significant burden on educators and leaving students who wish to practise independently with a limited and often static pool of exercises.

This project addresses this gap by integrating generative AI with structured machine learning outputs. It presents a full-stack web application, internally referred to during development as Code Quiz, that harnesses the generative capabilities of large language models [1] to produce both multiple choice questions and coding challenges on demand, based on a topic or prompt provided by the user. Rather than returning a raw block of AI generated text, the platform enriches every question it produces with two additional layers of machine learning intelligence: a predicted set of algorithmic topic tags, and a predicted difficulty level. These enrichments are produced by two custom trained classifiers that operate on the text of each generated question, running silently in the background without adding meaningful latency to the user experience.

The motivation for this system comes from three recurring limitations in existing platforms. First, static question banks grow stale and do not adapt to the breadth or specificity of what a learner wishes to study. A student who wants to practise tree traversal problems on a Tuesday evening should not be limited to the same twenty or thirty problems that a textbook happens to cover, and an instructor who

wishes to set coursework on a newly introduced topic should not have to author every problem by hand. Second, even when generic AI tools such as ChatGPT are used to generate questions, the output is typically unstructured and requires manual curation before it can be deployed as a teaching resource. The questions come back as free text without structured options, without test cases, without tags, and without any indication of how challenging they are likely to be. Third, learners attempting self directed study with coding problems often have no objective signal about how difficult a problem is or what underlying concept it is testing, making it difficult to build a structured learning path from simpler problems to harder ones.

By combining generative AI with trained machine learning models, this platform attempts to address all three problems at once: questions are produced dynamically to any specification, they are automatically tagged with the algorithmic topics they assess, and they are labelled with a predicted difficulty so that learners can make informed choices about which problems to attempt. Crucially, the topic tagger and the difficulty predictor are not wrappers around a second call to a language model. They are lightweight scikit learn pipelines [2] trained on real data: a LeetCode problem corpus for the tag classifier [3] and the IBM Project CodeNet dataset for the difficulty classifier [4]. This decision was deliberate. Running a second LLM call per question would be slow, expensive, and would provide no reproducibility from one run to the next, whereas a serialised classifier loaded once at startup produces deterministic annotations in a few milliseconds per question.

1.2 Project Scope and Level

This is a level 8 project developed as a final year applied project in computing. It encompasses a range of technically demanding areas, including full-stack web development, integration with external AI APIs, local model inference, custom machine learning model training and deployment, containerised infrastructure, and a social feature set that includes a user accounts system with experience points and a friends network.

The scope is deliberately broad. The intent is not to build a research prototype that solves a single narrowly defined problem, but rather to deliver a functional, end to end product that a real user could interact with. The application is fully containerised using Docker [5], meaning it can be deployed in a reproducible way with a single command. The frontend is a single page React application [6] served through Nginx [7]. The backend is a Python FastAPI server [8] that coordinates AI generation, machine learning inference, and code execution. A PostgreSQL database [9] persists user data. The two trained classifiers are serialised as model artifacts and loaded at server startup so that inference imposes no disk read cost on request handling.

The project does not attempt to be a drop in replacement for professional learning management systems, nor does it compete with platforms such as LeetCode at scale. Its contribution is in demonstrating that a tightly integrated pipeline connecting generative AI with custom trained classifiers can enrich automatically produced educational content in a meaningful and technically sound way, and that this can be achieved within the constraints of a solo final year project using open source tooling throughout.

1.3 Objectives

The following objectives guided the development of this project and will each be addressed explicitly in the system evaluation chapter:

1. Design and implement a full-stack web application that allows users to generate multiple choice and coding challenge quizzes through a natural language prompt, with support for multiple AI backends, including a cloud hosted model (OpenAI) and a locally hosted model (Ollama running Llama 3.1).

2. Train a multi label topic tag classifier capable of predicting relevant algorithmic topic tags for a given coding question, using a real world dataset of labelled programming problems.

3. Train a difficulty classifier capable of predicting whether a coding problem is easy, medium, or hard, in the absence of explicit human difficulty labels, by engineering a meaningful proxy label from available submission data.

4. Integrate both trained classifiers into the backend inference pipeline so that every generated question is automatically enriched with predicted tags and a predicted difficulty level before being returned to the user.

5. Implement a code execution engine within the backend capable of running user submitted code against test cases across multiple programming languages, and returning pass or fail results per test.

6. Deliver a user accounts system with authentication, profile management, an experience point and levelling mechanic, and a social friends network.

The primary metric for evaluating the machine learning components is classification performance on held out test sets, using precision, recall, and the F1 score. For the topic classifier specifically, F1 micro and F1 macro will both be reported, as they each reveal different aspects of performance when class imbalance is present:

F1 micro is dominated by the most populous classes, whereas F1 macro treats every class equally regardless of support. For the difficulty classifier, F1 macro is the primary metric given the three class setup with roughly balanced support. For the application itself, the evaluation focuses on whether the pipeline functions correctly, whether generated questions receive sensible tags and difficulty labels, and whether the code execution engine correctly judges test case outcomes across every supported language.

1.4 Chapter Overview

The remainder of this dissertation is structured as follows. Chapter 2 describes the development approach, the planning and requirements work done upfront, the GitHub based workflow, the continuous integration pipeline, and the validation strategies for both software and machine learning components. Chapter 3 surveys the technologies, libraries, frameworks, and models that underpin the system, covering large language models, the machine learning techniques used in both classifiers, the web technologies used for the frontend and backend, containerisation, and the training datasets. Chapter 4 presents the architecture of the application, including component diagrams, data flow descriptions, the database schema, the full training pipelines, and the design of the inference bridge between the backend and the trained model artefacts. Chapter 5 assesses the project against the objectives, presenting classification metrics, confusion matrices, per class breakdowns, and an honest examination of limitations. Chapter 6 concludes with a summary of outcomes, unexpected insights that emerged, and concrete opportunities for future investigation.

1.5 Project Repository

The full source code, including the frontend, backend, training scripts, Docker configuration, and serialised model artefacts, is available at the following repository:

`https://github.com/fionnmcgoldrick123/programming-quiz-app`

A brief screencast demonstrating the application is also available via the repository README.

The repository contains a `frontend` directory (React and TypeScript source), a `backend` directory (FastAPI application with route handlers, services, auth, and the code execution engine), an `ml_models` directory (training scripts and serialised `.pkl` artefacts for both classifiers), a `docker-compose.yml` at the root that brings all three services up in a single step, and a `docs` directory containing the LaTeX source for this dissertation.

Chapter 2

Methodology

2.1 Development Approach

This project is primarily centred on the design and development of a full-stack software system. The main objective was to build and deliver a functional application, and the process was therefore guided by established software engineering practices. However, the development was supported by a focused research component, particularly in the areas of machine learning model selection and training, where decisions around algorithms, datasets, and evaluation strategies were informed by existing literature and exploratory experimentation before implementation. An empirical approach was applied throughout: rather than fixing a design upfront, each subsystem was developed, tested, measured, and refined based on observed performance.

The overarching development approach was agile, iterative, and incremental, drawing on the principles of the Agile Manifesto [10] and lightweight practice suggested by Beck and Andres [11]. Rather than attempting to design the entire system upfront and implement it in sequence from start to finish, the project was broken into a series of development cycles, each delivering a working vertical slice of functionality that could be tested, reviewed, and built upon. This approach was well suited to a project of this complexity, where the requirements for the machine learning components could not be fully specified in advance. The decision to use a multi label classifier for topic tagging, for example, only emerged once the nature of the training data was properly understood: the LeetCode corpus showed that problems routinely carry several tags at once, which ruled out a single label classifier from the outset. Similarly, the approach to difficulty labelling only became clear after exploring the structure of the IBM CodeNet dataset and noticing that while no explicit difficulty labels were available, acceptance rate provided a usable proxy signal.

2.2 Planning and Requirements

Before any implementation began, a significant amount of upfront planning was undertaken to establish the scope of the project and the technologies that would be used. This planning work was produced entirely within Figma and is documented in the `/docs/project_planning` folder of the project repository. The artefacts produced include a concept summary outlining the high level vision for the platform, an original system architecture diagram showing how the major components would relate to each other, a planned tech stack document justifying the selection of each major technology, prototype requirements describing the expected behaviour of the system at a minimum viable product level, and a Docker container plan specifying how the three services (database, backend, and frontend) would be orchestrated. A Gantt chart was also produced, created in Canva, mapping out the intended timeline for each phase of development from initial research through to final testing and writeup.

This planning phase served as a storyboarding exercise for the entire project. By working through the architecture visually before writing any code, it was possible to identify dependencies between components early. For instance, it became clear that the backend API contracts needed to be defined before meaningful frontend development could begin, and that the machine learning models needed to be trained and serialised before they could be integrated into the inference pipeline. Identifying these ordering constraints in advance meant that the sequence of sprints could be arranged to minimise rework.

Requirements were determined through a combination of domain analysis and personal specification. Since this was a solo project without an external client, the requirements were derived by examining similar platforms, identifying the gaps described in the introduction, and translating them into concrete functional and non functional specifications. Functional requirements included question generation via prompt, multiple choice and coding challenge formats, topic tag prediction, difficulty prediction, code execution with test case evaluation, user authentication, and a social friends system. Non functional requirements included containerised deployment for reproducibility, sub second machine learning inference latency so that the enrichment step would not degrade the perceived speed of question generation, and support for at least five programming languages in the code runner so that the platform would serve a realistic spread of coursework scenarios.

2.3 Iterative Development Cycles

Development proceeded through a series of loosely defined sprints, each targeting a coherent piece of functionality. The early sprints focused on establishing the

foundational infrastructure: configuring the PostgreSQL database, handling API calls to large language models, and building the basic FastAPI skeleton with a working database connection. Once the infrastructure was stable, subsequent cycles added the AI generation pipeline, the machine learning training and integration work, the code execution engine, the authentication system, and finally the social features and frontend polish.

This incremental approach meant that the system was in a runnable and testable state throughout most of the development period. At no point was the entire codebase non functional for more than a short period while a breaking change was integrated. This mattered because it allowed problems to be caught early, when the context of what had just changed was still fresh, rather than at the end of a long development phase when the source of a bug could be difficult to trace. This mirrors the principle behind continuous integration: smaller changes are easier to test and revert than large ones.

Where technical problems arose, an empirical approach was taken. When the topic tag classifier initially produced poor results, for example, the response was not to switch to a different algorithm immediately but rather to investigate the training data more carefully, examine the class distribution, experiment with threshold tuning, and try different feature engineering strategies before drawing conclusions. This reflects a scientific problem solving habit even within a software development context: observe, hypothesise, test, and iterate. In several cases the root cause turned out to be in the data rather than the model. Rare topic tags with only a handful of examples were producing unreliable predictions not because the classifier was wrong but because there simply was not enough signal to learn from. The eventual fix was a minimum sample threshold filter on the tag set rather than a change in algorithm.

2.4 GitHub Workflow

GitHub was used throughout the project as both the version control system and the primary project management tool. The repository followed a flexible branching strategy centred around a stable `main` branch, with additional branches created as needed to support ongoing development. Rather than adhering to a strict two branch model, new branches were often created to address specific features or areas of the system, such as social functionality, or to work on tasks associated with particular GitHub issues. This lightweight approach suited a solo project, where the overhead of maintaining long lived parallel branches would have outweighed the benefits.

Work was tracked using GitHub Issues, where each issue represented a discrete unit of work, including new features, bug fixes, refactoring tasks, or research

activities. Issues were organised using a set of labels tailored to the project, including categories such as classifier model, bugs, coding sandbox, and MCQ quiz functionality, alongside default labels. This labelling approach allowed for efficient filtering and provided a clear overview of the types of work remaining at any stage of development.

Milestones were used to group related issues into time bounded delivery targets aligned with the overall project plan. Each milestone corresponded to a major phase of development, such as the MVP launch, MCQ system completion, classifier integration, and final dissertation submission. This structure provided a high level view of progress and helped ensure that development remained aligned with key deadlines, including the Christmas seminar presentation and the April submission.

When beginning work on a task, a new branch was typically created, sometimes linked to a specific issue, and named in a way that reflected the functionality being developed, for example `social_features` or `code_sandbox_v2`. Development was carried out through incremental commits with descriptive messages. Upon completion, changes were merged back into the main codebase. This workflow maintained a clear and navigable project history while supporting parallel development across different features, and it left a commit trail that serves as evidence of sustained effort across the full development period rather than a last minute sprint.

2.5 Continuous Integration Pipeline

A continuous integration pipeline was configured using GitHub Actions [12] and is defined in the `.github/workflows` directory of the repository. The pipeline triggers on every push to the `main` branch and on every pull request targeting it. This ensured that no code could be merged into the `main` branch without first passing automated checks.

The pipeline is divided into three parallel jobs. The frontend job installs Node.js 22 and the project dependencies, runs ESLint to check for code quality issues, performs a TypeScript type check using `tsc -noEmit` to catch type errors without producing output files, and then runs the production build using Vite [13] to confirm that the application compiles successfully. Passing build artefacts are uploaded for a retention period of seven days. The backend job sets up Python 3.11, runs the Black formatter in check mode to verify that the code conforms to the project's formatting standard, runs Flake8 to catch syntax errors and undefined name references, and compiles every Python file in the backend directory to validate syntax. The ML models job runs a similar set of checks across the `ml_models` directory. A final summary job runs only if all three parallel jobs succeed, providing a clear single status indicator for the overall health of the pipeline.

This setup served two important purposes. First, it enforced a baseline level

of code quality automatically, removing the need for manual checks on every change and catching regressions immediately. Second, it provided confidence when merging feature branches that the integration would not break the build. The use of `continue-on-error: true` on the formatting and complexity checks meant that these were informational rather than blocking, allowing the pipeline to surface stylistic feedback without preventing progress when minor formatting divergences were present. This balance is important, as overly strict pipelines tend to be ignored, while meaningful failures remain useful signals.

2.6 Validation and Testing

Testing was carried out primarily through manual integration testing and through the empirical evaluation of the machine learning models on held out test sets. Given that this was a solo project and the focus of the evaluation chapter is on measurable outcomes rather than on test coverage metrics, a full unit test suite was not implemented. A representative sample of pytest tests does exist under `backend/tests`, covering core logic such as JWT token round tripping and password hashing, but the bulk of validation was done at the integration level, where the behaviour that matters to the end user is actually observable. The continuous integration pipeline's syntax validation and build checks served as a lightweight automated safety net across all three components of the codebase.

The machine learning models were evaluated rigorously using train, validation, and test splits. For the topic tag classifier a sixty, twenty, twenty split was used, with the validation set serving as the basis for per label decision threshold optimisation before final evaluation on the held out test set. The rationale for this approach is set out in detail in the System Design chapter, but the key point is that tuning thresholds on the test set would have leaked information and produced optimistic estimates of generalisation. Holding out a separate validation set for threshold selection, then reporting test set numbers only once at the end, is the standard cross validation protocol described by Kohavi [14]. For the difficulty classifier a five fold cross validation strategy was used during hyperparameter search via `RandomizedSearchCV` [15], which is a more computationally efficient alternative to exhaustive grid search when the hyperparameter space is large.

The code execution engine was tested manually by submitting known correct and intentionally incorrect solutions across each supported language and verifying that the test case evaluation logic returned the expected outcomes. The end to end question generation pipeline was tested by issuing prompts and inspecting the enriched responses, checking that the tags and difficulty labels returned were plausible for the generated content. Where prompts produced questions that the model then tagged implausibly, those cases were logged and fed back into a later

round of prompt engineering or threshold tuning, which over time reduced the rate of surprising outputs.

2.7 Technology Selection Criteria

The choice of specific technologies was guided by a small number of consistent criteria. First, each major technology had to be open source or freely available at the project's scale, because a solo student project cannot justify paid tooling beyond what is strictly necessary. Second, each technology had to have active community support and documentation sufficient to resolve problems without a commercial support contract. Third, where a choice was made between two roughly equivalent options, the option with stronger type safety, stronger validation, or stronger reproducibility was preferred, on the grounds that these properties compound favourably as a codebase grows. FastAPI over Flask, TypeScript over plain JavaScript, and Pydantic validation on every request boundary are all choices that reflect this preference.

Chapter 3

Technology Review

3.1 Overview

This chapter provides a detailed examination of the technologies, tools, frameworks, datasets, and algorithmic approaches that underpin the system described in this dissertation. It is organised to mirror the architecture of the project itself, moving from the AI generation layer through the machine learning components, down to the backend infrastructure, database, and frontend. Where possible, references are drawn from peer reviewed publications, official standards bodies, and authoritative technical documentation. The purpose of this chapter is to demonstrate that each technology choice was informed by research and that the overall system design is grounded in established principles from both software engineering and applied machine learning. Each section tries to explain what the technology is, why it was chosen for this project, and where possible how it compares to the alternatives that were considered.

3.2 Large Language Models and Generative AI

3.2.1 The Emergence of Large Language Models

The capacity of this platform to generate quiz questions and coding challenges on demand rests on large language models: neural networks trained on enormous text corpora, capable of producing coherent output in response to a natural language prompt. The architectural basis is the Transformer introduced by Vaswani et al. [16], which replaced the recurrent and convolutional structures of earlier sequence models with a purely attention based mechanism. Attention allows every position in a sequence to attend directly to every other position, removing the sequential bottleneck that had limited previous architectures and making it practical to train

far larger models.

Subsequent work established that scaling these models produced qualitative capability improvements. The bidirectional BERT model of Devlin et al. [17] showed that pre training on unlabelled text followed by task specific fine tuning could set new state of the art results across natural language understanding benchmarks. The GPT line scaled autoregressive decoder only models instead, and the work of Brown et al. [1] on GPT 3 demonstrated that scaling to 175 billion parameters produced few shot and zero shot task performance, meaning these models could complete tasks from a short description alone. This established the prompt based interaction pattern this project relies on: a structured prompt is sent to the model describing the desired output, and the model generates a conforming response.

The project's primary backend is `gpt-4o-mini`, accessed through the OpenAI API, a smaller, cost efficient member of the GPT 4 family suited to reliable instruction following. A secondary backend using Ollama [18] with the `llama3.1:8b` model, descended from Meta's LLaMA work [19], is also supported, enabling locally hosted generation without reliance on an external API. Abstracting both backends behind a common interface in `ai_models.py` means the system is not locked into any single provider.

3.2.2 Prompt Engineering

Since LLMs are prompted rather than fine tuned in this project, output quality depends heavily on prompt construction, a practice known as prompt engineering [20]. The system uses two prompt templates (`prompt_guide.txt` for MCQ generation and `coding_prompt_guide.txt` for coding challenges), each specifying a strict JSON output schema to constrain the model to produce parseable structured responses. This draws on the established practice of using examples and formatting constraints in prompts, as described by Brown et al. [1]. Recent work on chain of thought prompting by Wei et al. [21] has shown that intermediate reasoning steps can improve output on multi step tasks; the structured schema used here serves as a lighter weight output scaffold.

A known limitation of prompt based integration is that LLMs do not always produce strictly valid JSON. OpenAI has since introduced a structured output mode [22] that enforces schema conformance at the API level. This was not adopted at the time of development but is identified as a clear improvement in the Conclusion.

3.3 Machine Learning Foundations

3.3.1 Text Representation with TF IDF

Both machine learning classifiers in this project use term frequency inverse document frequency, or TF IDF, as the primary method of transforming text into numerical feature vectors. TF IDF was introduced by Spärck Jones [23] in 1972 as a term weighting scheme for information retrieval. The core insight is that a term's importance to a document is proportional to how often it appears in that document, but inversely proportional to how often it appears across the entire corpus. Common words like `the` or `and` receive low weights because they appear in almost every document and therefore carry little discriminative information. Rare, domain specific terms receive high weights because their presence is more informative. Salton and Buckley [24] provided a systematic comparison of alternative weighting schemes and established the form of TF IDF that is now standard, and Ramos [25] offers a more accessible treatment suited to applied settings.

Formally, for a term t in a document d within a corpus D, the TF IDF weight is defined as:

$$\text{tf idf}(t, d, D) = \text{tf}(t, d) \times \log\left(\frac{|D|}{|\{d \in D : t \in d\}|}\right)$$

This project uses scikit learn's `TfidfVectorizer` [2] with sublinear term frequency scaling, which replaces the raw term frequency $\text{tf}(t, d)$ with $1 + \log(\text{tf}(t, d))$ to reduce the dominance of very frequent terms within a single document. Two separate vectorisers are combined using `FeatureUnion`: a word level vectoriser capturing unigrams through trigrams, and a character level vectoriser operating on three to five character n grams. The character level component is particularly valuable for code adjacent text, where tokens such as `O(n)`, `dp[i]`, and `arr[j]` form meaningful patterns that a word level tokeniser would either split or discard. A word tokeniser typically treats punctuation as a boundary and therefore cannot learn that `dp[i]` is a recognisable motif, whereas a character n gram vectoriser will naturally capture `dp[` and `]` as features.

3.3.2 Logistic Regression

The topic tag classifier uses logistic regression as its core estimator, implemented within scikit learn [2, 26]. Logistic regression is a discriminative linear classifier that models the probability of class membership as a sigmoid function of a linear combination of input features. For a feature vector $\mathbf{x}$ and weight vector $\mathbf{w}$:

$$P(y = 1 \mid \mathbf{x}) = \sigma(\mathbf{w}^T\mathbf{x} + b) = \frac{1}{1 + e^{-(\mathbf{w}^T\mathbf{x}+b)}}$$

Parameters are estimated by maximising the log likelihood of the training labels under this model, with L2 regularisation controlled by the hyperparameter C to prevent overfitting. The `class_weight= 'balanced'` setting was used to adjust class weights inversely proportional to their frequency in the training data, a standard technique for addressing class imbalance [27]. Logistic regression was chosen over more complex models for this component because TF IDF features tend to be high dimensional and sparse, and linear classifiers are well known to perform competitively in this regime [28]. Deeper models such as transformer based encoders would likely yield better absolute performance, but at significantly higher inference cost, which would defeat the design goal of sub second enrichment per question.

3.3.3 Multi Label Classification

Because a single coding problem can belong to multiple algorithmic categories simultaneously (for example dynamic programming and binary search together), topic tagging is inherently a multi label classification problem [29]. This project uses scikit learn's `OneVsRestClassifier`, which trains one independent binary classifier per label. The `MultiLabelBinarizer` transforms the list of string tags into a binary vector of length equal to the unique label count [2]. An alternative, the classifier chain of Read et al. [30], was also evaluated but did not improve on the simpler one versus rest approach on the LeetCode corpus.

The decision threshold was tuned per label on the validation set rather than using a default 0.5 for every label. Lipton et al. [31] show that the F1 maximising threshold depends on the positive class base rate and cannot be assumed constant. The system uses a shrinkage regularised variant in which the per label optimum is blended with a single global optimum to prevent overfitting on rare labels where the validation sample is small.

3.3.4 Gradient Boosting for Difficulty Prediction

The difficulty classifier uses ensemble methods, comparing Random Forest [32], HistGradientBoosting, and Gradient Boosting estimators from scikit learn [2] with XGBoost [33]. Gradient boosting, as formalised by Friedman [34], builds an ensemble of shallow decision trees by fitting each successive tree to the residuals of the previous ensemble. The `HistGradientBoostingClassifier` bins continuous features into discrete histograms before splitting, dramatically reducing training time and handling missing values natively. Linear SVMs [35] were also evaluated as a baseline. Model selection used five fold cross validation with `RandomizedSearchCV` [15], scoring on F1 macro to ensure balance across difficulty classes. Random search is preferred to grid search for larger hyperparameter spaces because, as Bergstra

and Bengio show, it explores more distinct values of the important hyperparameters for the same computational budget.

3.3.5 Proxy Labels and Class Imbalance

The CodeNet dataset [4] contains no human assigned difficulty labels, so this project uses acceptance rate as a proxy, binning problems into terciles: the top third labelled easy, the middle medium, and the bottom hard. Using proxy labels is an established technique in weakly supervised learning [36]. A noisy, cheap to obtain label is substituted for an expensive authoritative one, and the model is trained to recover the underlying signal despite the noise.

Class imbalance is a more serious issue for the topic classifier. The LeetCode corpus is severely imbalanced: the Array tag appears on more than a thousand problems while Depth First Search appears on fewer than thirty. Japkowicz and Stephen [27] review the common mitigations, including cost sensitive learning, oversampling, and undersampling. The `class_weight='balanced'` setting used in the logistic regression base classifier is an instance of cost sensitive learning. Synthetic oversampling via SMOTE [37] was considered but not adopted, as its interaction with high dimensional sparse TF IDF vectors is not straightforwardly meaningful.

3.4 Datasets

3.4.1 LeetCode Dataset for Topic Tag Classification

The topic tag classifier was trained on a LeetCode derived dataset obtained from Kaggle [3], distributed as a CSV file containing problem titles, descriptions in HTML format, and associated topic tags. LeetCode is a widely used competitive programming and interview preparation platform, and its problems have well established community assigned tags such as Array, Hash Table, Dynamic Programming, and Two Pointers. After cleaning, parsing multi value tag strings, and filtering out tags appearing in fewer than sixty examples (to remove labels with insufficient training signal), this dataset provided a realistic and domain appropriate training corpus for the tagging task. An initial exploratory analysis on the filtering threshold established that too low a threshold admitted labels where the classifier could not learn any useful pattern, while too high a threshold collapsed the problem into a handful of very common tags that were already trivially easy to predict.

3.4.2 IBM Project CodeNet

The difficulty classifier was trained on IBM Project CodeNet [4], published by Puri et al. at NeurIPS 2021. CodeNet is a large scale dataset containing approximately fourteen million code samples across 4,053 competitive programming problems in fifty five programming languages. Each problem submission is annotated with metadata including its acceptance status, CPU runtime, memory usage, and code size. The dataset was constructed from two online judging platforms and has been used widely in research on AI for code.

For this project, per problem statistics were aggregated from individual submission CSV files, and problem descriptions were parsed from HTML to extract lexical and structural features. The scale and richness of CodeNet's metadata made it the most suitable publicly available dataset for engineering the numeric and textual features required by the difficulty model. The dataset is released with a permissive licence, which was a practical consideration for a final year project: no restrictive terms constrain how the derived model artefacts are used or distributed.

3.5 Backend Framework: FastAPI

The backend is built using FastAPI [8], a Python web framework released in 2018 by Sebastián Ramírez, layered on Starlette [38] for asynchronous request handling and Pydantic [39] for data validation. Routes declare the types of their parameters and return values using Python type hints, which FastAPI uses to automatically validate incoming requests, serialise responses, and generate interactive OpenAPI documentation. Independent benchmarks place FastAPI among the fastest Python frameworks, comparable in throughput to Node.js and Go when run under the Uvicorn ASGI server [8].

FastAPI was chosen over Flask and Django REST Framework for two reasons. Its native async support allows the backend to handle concurrent AI API calls without blocking, which matters because AI generation latency dominates overall request time. Automatic Pydantic validation removes a large category of defensive checking code from route handlers. Lathkar's book [40] provides a comprehensive treatment of the framework's async architecture. All request and response structures are defined as Pydantic `BaseModel` subclasses, so malformed input is caught at the API boundary and never reaches business logic, a pattern of edge validation that reduces defensive code and eases diagnosis of failures [41].

3.6 Database: PostgreSQL

PostgreSQL [9] is used as the relational database, accessed through `psycopg` version 3 with a `dict_row` factory. PostgreSQL is a mature, open source, object relational database with roots in the POSTGRES project at UC Berkeley [42], ACID compliant since 2001 and implementing multi version concurrency control. The schema stores user accounts, profiles, quiz results, experience and levelling data, and the social graph of friendships. A relational database was chosen over a document store because the social features involve many to many relationships with status qualified edges, which benefit from normalised modelling and join based querying. Native PostgreSQL features used include `TEXT[]` array columns (for topic tags on quiz results), `JSONB` columns (for full quiz session content), and `CHECK` constraints (to enforce at the database level that a user cannot friend themselves).

TablePlus [43] was used throughout development as the primary graphical interface for inspecting and managing the PostgreSQL database. As a native GUI client, it supports direct connection to PostgreSQL and provides inline editing of rows and table structure, as well as advanced filtering to quickly locate specific records. During development, TablePlus served as the main tool for verifying schema correctness, manually inspecting table contents after test runs, and executing ad hoc queries to debug application behaviour. Its built-in SQL editor with syntax highlighting and the ability to split query results across multiple tabs made it straightforward to cross-reference data across the user accounts, quiz results, and friendships tables simultaneously. The code review feature and snapshot capability also provided a safety net when making direct modifications to the development database, ensuring unintended changes could be identified before being committed.

3.7 Containerisation: Docker and Docker Compose

The entire application is containerised using Docker [5]. Docker packages applications and their dependencies into lightweight containers that run as host processes isolated via Linux kernel namespaces and control groups. Its key promise, that a containerised application behaves identically wherever it runs, matters for a project of this complexity, where the backend depends on a specific Python version, Node.js for parts of the code execution engine, and pre trained model artefacts. Docker Compose [44] orchestrates multiple containers declaratively in a single YAML file. Three services are defined: PostgreSQL, the Python backend, and the React frontend served by Nginx. The backend Dockerfile installs Python 3.11 slim with Node.js 22 alongside for the JavaScript and TypeScript runtimes. The frontend uses a two stage build: Node 22 alpine produces the Vite artefact, which

is served by Nginx stable alpine with single page fallback routing, gzip compression, and one year cache headers on hashed assets.

3.8 Frontend: React and TypeScript

The frontend is built using React [6] and TypeScript [45], compiled by Vite [13]. React is an open source JavaScript library for component based interfaces, developed at Meta and open sourced in 2013. Its central innovation is the virtual DOM: rather than directly manipulating the browser DOM on every state change, React maintains an in memory UI representation and computes the minimal set of changes required to synchronise the real DOM with the current state. TypeScript adds a static type system to JavaScript, catching type errors at compile time. For a project of this size, where data flows from an AI model through a parser, a machine learning enrichment layer, and a REST API before reaching the frontend, type safety at the boundary protects against category mismatches: a Pydantic schema on the backend and a matching TypeScript interface on the frontend form a contract the compiler can check. Vite provides near instant hot module replacement and produces optimised production bundles. State is managed using React hooks and an `AuthContext` provider that exposes the JWT and user session. The Monaco Editor [46], the same engine that powers Visual Studio Code, is embedded in the coding sandbox to provide syntax highlighting, auto indentation, and language aware features for all five supported languages.

3.9 Authentication: JSON Web Tokens and bcrypt

User authentication uses JSON Web Tokens as standardised in RFC 7519 [47]. A JWT is a compact URL safe representation of claims encoded as JSON and signed with HMAC SHA256. On login, the backend issues a signed token carrying the user id and an expiry timestamp; the client stores it and includes it in the Authorization header on subsequent requests, where the backend verifies the signature without a database lookup. This stateless pattern scales well because no server side session state is held. The PyJWT library handles encoding and decoding on the backend. Passwords are hashed with bcrypt [48], an adaptive hashing function based on Blowfish whose configurable cost parameter keeps it secure as attacker hardware improves, unlike a bare cryptographic hash such as SHA 256 which is fast by design and therefore easy to brute force. OWASP authentication guidance [49] recommends adaptive hashing functions such as bcrypt, scrypt, or argon2.

3.10 Code Execution Engine

The code execution engine supports Python, JavaScript, TypeScript, Java, and C#. For each submission, the backend writes the user's code to a temporary file and invokes the runtime via `subprocess.run` with a five second timeout. The backend Docker image installs Node.js 22 to support JavaScript and TypeScript, and both Java and C# runtimes are also provided. For test case evaluation, the engine compares actual standard output against the expected output per case. Subprocess isolation is a common pattern in online judges; it does not match kernel level isolation systems such as Isolate, but is appropriate for the scope and threat model of this project, where users are authenticated and have incentive to learn rather than to subvert the platform. Production deployment with untrusted users would need stronger isolation, flagged as future work.

3.11 Data Interchange and Continuous Integration

All frontend and backend communication uses JSON (RFC 7159 [50]), consistent with the REST architectural style [51]. FastAPI automatically generates an OpenAPI 3.0 specification [52] from the Pydantic model definitions, documenting every endpoint's request and response schemas. The continuous integration pipeline uses GitHub Actions [12] with workflows defined in `.github/workflows/`, running three parallel jobs covering the frontend (ESLint, TypeScript type checking, Vite build), the backend (Black, Flake8, Python syntax validation), and the machine learning models directory.

3.12 Scientific Computing Libraries

Beyond scikit learn [2, 26], the machine learning components use NumPy [53] for array operations, pandas [54] for tabular data manipulation, and joblib [55] for serialising trained pipelines to `.pkl` files. The `ColumnTransformer` and `FeatureUnion` classes build modular preprocessing pipelines that apply different transformations to different feature subsets and concatenate the resulting matrices before model fitting. This pipeline based approach ensures preprocessing is consistently applied at inference time using the parameters fitted on training data, eliminating training and serving skew.

Hastie, Tibshirani, and Friedman [28] provide the statistical learning foundation for the estimators used, and Goodfellow, Bengio, and Courville [56] offer a reference should future iterations replace TF-IDF with a learned representation.

The `ColumnTransformer` class from scikit-learn [2] deserves particular attention because it is the component that makes it possible to apply fundamentally different transformations to different feature subsets within a single unified pipeline. In the difficulty classifier, the training matrix contains both dense numeric features derived from submission statistics and a single raw text column representing the problem description. Without `ColumnTransformer`, these would need to be preprocessed separately and then concatenated manually at each stage of experimentation, introducing opportunities for index misalignment and inconsistent imputation. By declaring a numeric branch (`SimpleImputer` followed by `StandardScaler`) and a text branch (`FeatureUnion` of two `TfidfVectorizer` instances) inside a single transformer, the entire preprocessing graph is captured in one serialisable object. When `joblib.dump` [55] writes the trained pipeline to disk, the fitted vocabulary, IDF weights, imputation medians, and scaling parameters are all preserved together. When the same object is loaded at inference time by `difficulty_service.py`, there is no possibility of applying the wrong vocabulary or the wrong scaling factors, because there is no separate step where they could diverge.

The `joblib` library itself warrants more than a passing mention. Its `dump` and `load` functions use a protocol based on Python's `pickle` serialisation format but with additional support for memory mapping large NumPy arrays [53]. For the difficulty model, which includes a fitted `XGBoost` [33] ensemble with hundreds of trees alongside the full TF-IDF vocabulary matrices, the serialised `.pkl` file is a non-trivial binary artefact. The memory mapping capability means that on systems with sufficient memory, the array components of the loaded model can be accessed without a full copy into RAM, which reduces the startup overhead of loading the model into the Uvicorn worker process. In practice, for the dataset sizes involved in this project, the full load takes under two seconds on modest hardware and is therefore not a bottleneck. However, the architectural decision to load the model once at module import time rather than on every request means this cost is paid exactly once per server process lifetime, regardless of how many quiz generation requests subsequently arrive.

The `pandas` [54] library plays a narrower but equally necessary role. The difficulty classifier's inference function, `predict_difficulty_for_question`, must construct a single row `DataFrame` whose column names exactly match those seen during training. This is the canonical way to pass data to a scikit-learn pipeline that contains a `ColumnTransformer`, because the transformer selects its input columns by name rather than by position. Using a raw NumPy array would require the caller to know and maintain the exact column ordering, which is fragile and error prone. The `DataFrame` interface makes the column names explicit at the call site, so any mismatch between the feature names produced by inference code and those expected by the trained transformer raises an immediate and informative error

rather than silently producing a wrong prediction.

Chapter 4

System Design

4.1 Overview

This chapter describes the concrete design of the AI powered quiz and coding challenge platform. It builds directly on the technology choices justified in the Technology Review, demonstrating how those choices were applied to produce a coherent, working system. The chapter is organised from coarse to fine grain: the overall architecture is presented first, followed by detailed descriptions of each major subsystem, the database schema, the machine learning training pipelines, the frontend architecture, and the deployment model. Diagrams are used throughout to augment the written explanation.

The system consists of three containerised services orchestrated by Docker Compose [5, 44]: a PostgreSQL 16 database, a Python FastAPI backend, and a React and TypeScript frontend served by Nginx [7]. These three services are loosely coupled through well defined interfaces. The backend exposes a REST API [51] conforming to the OpenAPI 3.0 specification [52], and the frontend communicates exclusively through that API using JSON [50] over HTTP. No service depends on the internal state of any other service, and the only inter service contracts are the REST API and the SQL schema. The three services and their communication paths are illustrated in Figure 4.1.

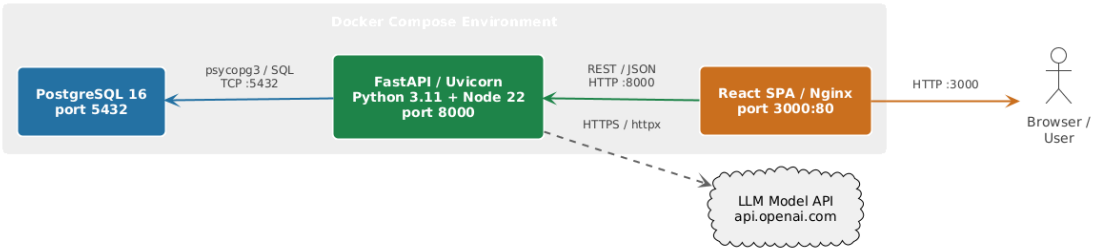

Figure 4.1: High level system architecture showing the three Docker Compose services and their communication paths.

4.2 Docker Compose Infrastructure

The deployment unit of the project is a Docker Compose file declaring three services: `db`, `backend`, and `frontend`. Each runs in an isolated container but communicates with the others over Docker's internal bridge network. Environment variables such as database credentials, the OpenAI API key, and the JWT secret are injected at runtime from a `.env` file and never committed to version control.

The `db` service runs the official `postgres:16-alpine` image with a named Docker volume `pgdata` mounted at `/var/lib/postgresql/data` for persistence across container restarts. Health checks via `pg_isready` combined with `depends_on: condition: service_healthy` ensure the backend does not start before the database is ready. The backend container is built from a custom Dockerfile based on `python:3.11-slim` with Node.js 22 installed alongside to support the JavaScript and TypeScript runtimes required by the code execution engine. The two serialised model artefacts, `topic_model.pkl` and `difficulty_model.pkl`, are copied in at build time. The application is served by Uvicorn [40] listening on port 8000. The frontend Dockerfile uses a two stage build: in the first, `node:22-alpine` runs `npm run build` to produce a Vite [13] static bundle; in the second, the compiled assets are copied into an `nginx:stable-alpine` image whose `nginx.conf` handles single page application fallback routing, gzip compression, and one year cache headers on content hashed assets.

4.3 Backend Architecture

The backend is a FastAPI application [8] whose entry point is `main.py`. FastAPI's `lifespan` context manager is used to run `init_db()` at startup, which creates the four database tables if they do not already exist, using idempotent `CREATE TABLE`

`IF NOT EXISTS` and `ALTER TABLE ... ADD COLUMN IF NOT EXISTS` statements so that migrations on an existing database are safe. The application is structured around four concerns: AI model integration, machine learning inference, code execution, and user management, which includes authentication, gamification, and social features. The call relationships between these modules are shown in Figure 4.2.

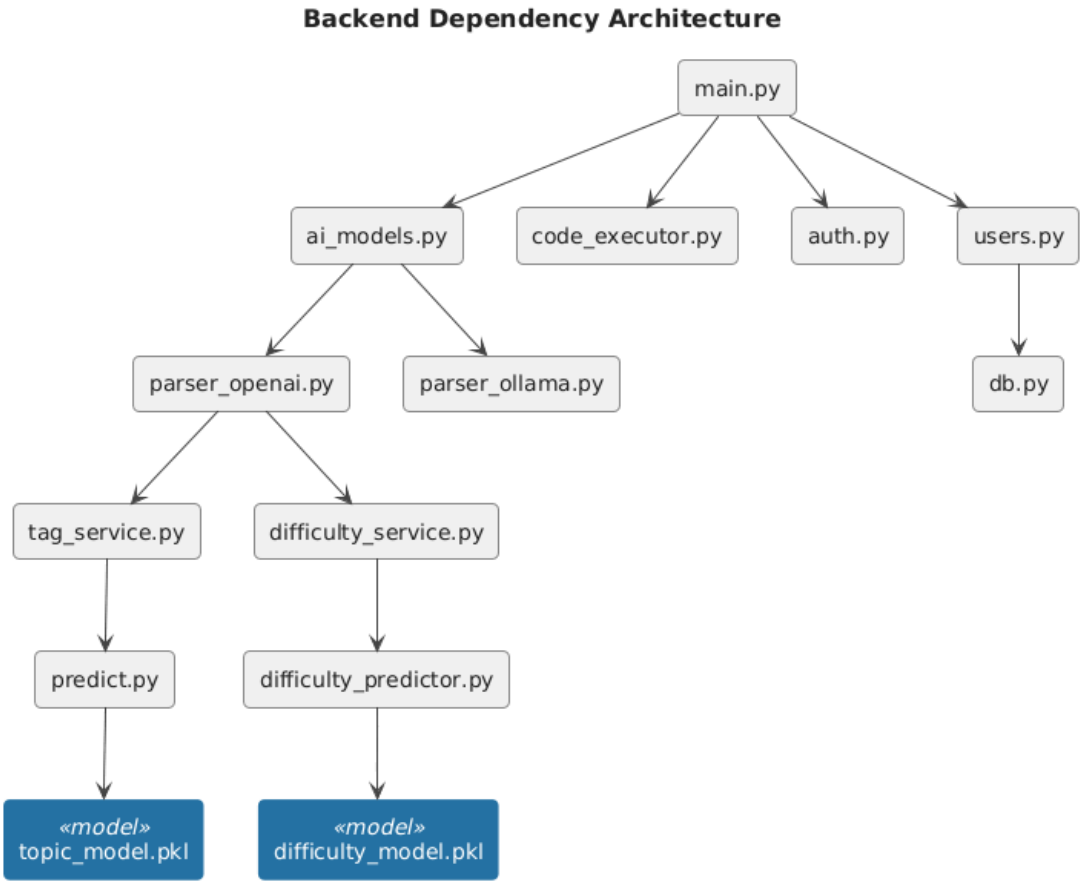

Figure 4.2: Backend module dependency diagram showing call relationships between the FastAPI entry point, service modules, AI parsers, and machine learning inference wrappers.

4.3.1 API Routes

The FastAPI application exposes the following groups of HTTP endpoints. All request and response bodies are validated against Pydantic `BaseModel` subclasses

defined in `pydantic_models.py` [39], which means that malformed or type incorrect input is rejected at the API boundary with a descriptive 422 error before it reaches any business logic.

- `POST /prompt`: accepts a `PromptRequest`, which bundles the user prompt, the quiz type, the target language, and the number of questions; invokes the AI generation pipeline; and returns a list of enriched `QuizSchema` or `CodingQuestionSchema` objects.
- `POST /run-code`: accepts source code and a language identifier, executes the code in a sandboxed subprocess, and returns stdout and stderr.
- `POST /submit-code`: accepts source code, a language identifier, and a JSON array of test cases; runs the code against each test case; and returns a per case pass or fail result.
- `POST /model`: toggles the active AI backend between OpenAI and Ollama.
- `POST /register` and `POST /login`: user registration and authentication, returning a signed JWT on success.
- `GET /verify-email` and `POST /resend-verification`: support the email verification flow that gates first time login.
- `GET /me`, `PATCH /me/profile`: retrieve and partially update the authenticated user's profile fields.
- `POST /add-xp`, `POST /save-quiz-result`: award experience points and persist a completed quiz result.
- `GET /user-stats`, `GET /quiz-history`: retrieve aggregated statistics and recent quiz sessions for the authenticated user.
- `GET /users/search`, `GET /users/{id}/profile`, `GET /users/{id}/stats`: search for users and retrieve public profiles and friends only statistics.
- `POST /friends/request/{id}`, `POST /friends/respond`, `GET /friends/requests`, `GET /friends/requests/count`, `GET /friends`, `GET /friends/count`, `DELETE /friends/{id}`: the full social friendship lifecycle.
- `GET /`, `GET /health`: root and server status utility endpoints.

4.3.2 AI Generation Pipeline

The AI generation pipeline is the most complex subsystem in the backend. The active AI backend is tracked by a global `current_model` variable in `ai_models.py`, toggled via `POST /model`. Both backends use `httpx.AsyncClient` with a 120 second timeout and await the full response before parsing begins, which simplifies the parser logic at the cost of not being able to show incremental output.

Once the raw response arrives, the appropriate parser (`parser_openai.py` or `parser_ollama.py`) extracts the JSON content from the model's reply. For MCQ quizzes, each question object is passed to `predict_tags_for_question(title, question)` to obtain predicted topic tags. For coding quizzes, `compute\_coding\_metadata()` is called first to extract numeric and lexical features (character count, word count, constraint language, code tokens), and these features are passed to `predict_difficulty_for_question()` alongside the question's AI generated metadata to obtain a difficulty label. Both enrichments are attached to the schema object before it joins the output list. The Ollama parser additionally strips option prefix characters such as `"A: "` from MCQ options, and the OpenAI backend wraps parsing in a three attempt retry on `json.JSONDecodeError`, `KeyError`, or `ValueError`.

4.3.3 Machine Learning Inference Pipeline

Model loading. Both machine learning models are loaded into memory as in process singletons. They are deserialised from their `.pkl` artefacts once and held in module level variables for the lifetime of the Uvicorn process. The difficulty model (`difficulty_model.pkl`) is loaded eagerly at module import time in `difficulty_service.py`. The `_load_model()` function is called at module level, outside any function body, so the `joblib.load` [55] call executes when Uvicorn first imports the module during startup. The topic model (`topic_model.pkl`) is loaded lazily: `_load_model()` is called inside `predict_topics()` and returns immediately if the module level `_pipeline` variable is already populated. This means the topic model is deserialised on the first quiz generation request after startup, then retained in memory for all subsequent requests. Both strategies ensure that the inference cost per request is limited to a single `pipeline.predict_proba()` call and a threshold comparison, which is negligible compared to the AI generation latency.

A compatibility fix is applied when loading the difficulty model: the helper functions defined in the `__main__` namespace at training time are re-injected into `sys.modules['__main__']` before `joblib.load` is called, preventing an `AttributeError` during deserialisation.

Topic tag prediction. The topic tag classifier is a scikit learn

`Pipeline` [2, 26] wrapping a `FeatureUnion` of two `TfidfVectorizer` instances (word level unigram to trigram and character level three to five n grams) followed by a `OneVsRestClassifier` wrapping a `LogisticRegression` estimator with balanced class weights. At inference time, the `predict_topic_from_parts(title, description, top_n=5)` function concatenates the title and description with a period separator, calls `pipeline.predict_proba()` to obtain a probability score for each of the trained labels, applies the per label optimised thresholds (stored in the `.pkl` bundle alongside the pipeline and the `MultiLabelBinarizer`), and returns up to five tags sorted by descending probability. If no label exceeds its threshold, the label with the highest probability is returned anyway, guaranteeing that every generated question carries at least one meaningful topic annotation.

Difficulty prediction. The difficulty classifier is a scikit learn `Pipeline` wrapping a `ColumnTransformer` that applies separate preprocessing paths to numeric and text features, followed by the best performing ensemble estimator selected during training. The numeric path applies `SimpleImputer(strategy='median')` followed by `StandardScaler` to the numeric features derived from submission statistics and problem description analysis. The text path applies a `FeatureUnion` of word level and character level `TfidfVectorizer` instances to the raw description text. At inference time, `predict_difficulty_for_question(schema)` receives a `CodingQuestionSchema` object, maps its fields to the column names the model expects, calls `extract_description_features()` to derive the text and keyword features from the question body, assembles a single row pandas DataFrame [54], and calls `pipeline.predict()` to obtain one of the three labels: easy, medium, or hard. If the model is unavailable, for example if the `.pkl` file is missing from the image, the function falls back to returning `medium`.

4.3.4 Difficulty Classifier Training Pipeline

The difficulty classifier training pipeline, implemented in `ml_models/difficulty_classifier/difficulty_predictor.py`, merits a more detailed description. The dataset is constructed by iterating over the CodeNet [4] per problem submission CSV files and aggregating per problem statistics (mean CPU time, mean memory, mean accepted code size, and acceptance rate). Problems with fewer than ten submissions are dropped because their aggregates are statistically meaningless. Problem descriptions are extracted from the corresponding HTML files and parsed to produce a plain text form, a word count, a character count, a count of numeric constraint tokens, and a set of binary keyword flags indicating the presence of algorithmic vocabulary such as recursion, dynamic programming, graph, sort, search, string, math, greedy, stack, and complexity.

Labels are derived from acceptance rate by percentile binning: the top tercile by

acceptance rate is labelled easy, the middle tercile medium, and the bottom tercile hard. This produces a balanced three class dataset by construction. The resulting training matrix contains thirty four numeric features (base statistics, log transforms, ratios, and interaction terms) plus one text feature that is subsequently expanded into word and character n grams by the preprocessor. The split uses stratified sampling to produce a sixty, twenty, twenty train, validation, and test partition. Seven candidate models are trained using `RandomizedSearchCV` [15] with forty iterations and five fold cross validation, scoring on F1 macro. The best model, XGBoost [33] in the final run, is retrained on the combined train and validation sets and evaluated once on the held out test set. The full pipeline, including the `LabelEncoder`, is serialised to `difficulty_model.pkl` via joblib.

4.3.5 Topic Tag Classifier Training Pipeline

The topic tag training pipeline, in `ml_models/tag_classifier/train.py`, follows a similar structure but is adapted to the multi label nature of the problem. The LeetCode [3] CSV is loaded, rows with null descriptions or null tag lists are dropped, and the tag strings are parsed from their stored representation into lists. Tags appearing in fewer than sixty problems are filtered out, and any row with no remaining tags is dropped. A `MultiLabelBinarizer` converts the tag lists to binary indicator vectors and is saved alongside the model for use at inference.

The training matrix is a combined `cleaned\_title+'.'+cleaned\_description` string per problem. The feature pipeline is a `FeatureUnion` of a word level TF IDF vectoriser (up to fifty thousand features, unigrams through trigrams) and a character level TF IDF vectoriser (up to twenty thousand features, three to five character n grams). Seven candidate architectures are trained using GridSearchCV with three fold cross validation, scoring on F1 micro: logistic regression with word features only, logistic regression with combined word and character features, an SGD classifier with word features, a classifier chain of logistic regressions with combined features, Multinomial Naive Bayes, Complement Naive Bayes, Random Forest, k Nearest Neighbours, and Linear SVC. All models use a OneVsRestClassifier wrapper except the classifier chain, which handles label dependencies directly.

A two stage threshold optimisation is then performed on the validation set. Stage A finds a single global optimum threshold by sweeping from 0.15 to 0.70 in steps of 0.01 and choosing the value that maximises F1 micro. Stage B computes a per label optimum by sweeping each label independently from 0.10 to 0.80 in steps of 0.025. The final per label threshold is a blend of the per label optimum and the global optimum with a shrinkage factor of 0.5. This shrinkage is a conservative bet against overfitting: pure per label optimisation on a validation set with small support per label is vulnerable to noise, whereas pure global optimisation discards the benefit of knowing that rare labels need lower thresholds. The 0.5 blend was

chosen because it exactly halves the influence of the per label sample on the final threshold, which is a principled starting point when there is no stronger prior. The best model is then retrained on train plus validation combined and evaluated once on the test set, and the pipeline, the binarizer, the thresholds, and the class list are all serialised together in `topic_model.pkl`.

4.3.6 Code Execution Engine

The code execution engine is implemented in `services/code_executor.py` and runs inside the FastAPI process rather than as a separate service. For both `POST /run-code` and `POST /submit-code`, the engine writes the user's source code to a temporary file using Python's `tempfile` module, spawns the appropriate runtime as a child process using `subprocess.run()`, captures stdout and stderr, and deletes the temporary file in a `finally` block before returning. A five second timeout is enforced on every subprocess call. Submissions that exceed this limit are terminated and a timeout message is returned to the caller. The complete flow is shown in Figure 4.3.

The runtime invocation strategy varies by language. Python uses the `python` interpreter directly. JavaScript uses `node`. TypeScript is executed directly using Node.js 22 with the `-experimental-strip-types` flag, which removes type annotations at runtime and eliminates the need for a separate transpilation step. Java requires a two step process: `javac` compiles the source file to a `.class` file, then `java` executes it. C# uses the `dotnet` CLI. Node.js 22 is installed in the backend Docker image to support both JavaScript and TypeScript execution. The Python interpreter is provided by the base image.

For test case evaluation (`POST /submit-code`), the engine iterates over the list of test cases provided in the request body. For Python submissions, the function name is extracted via a regex, then a test harness is appended that calls the function with the supplied input arguments and prints a marked result line to stdout. The harness output is parsed and compared to the expected value. The full list of per case results is returned to the frontend for rendering.

4.3.7 Authentication and User Management

Authentication follows the stateless JWT pattern standardised in IETF RFC 7519 [47]. When a user registers, their password is hashed using bcrypt [48] with a generated salt before storage. On login, the stored hash is compared to the provided password using `bcrypt.checkpw()`. If it matches, a JWT is generated using PyJWT with the user's id as the subject claim and a twenty four hour expiry encoded in the `exp` claim. The token is signed using HMAC SHA256 with a secret loaded from the environment. The frontend stores the token in `localStorage` and

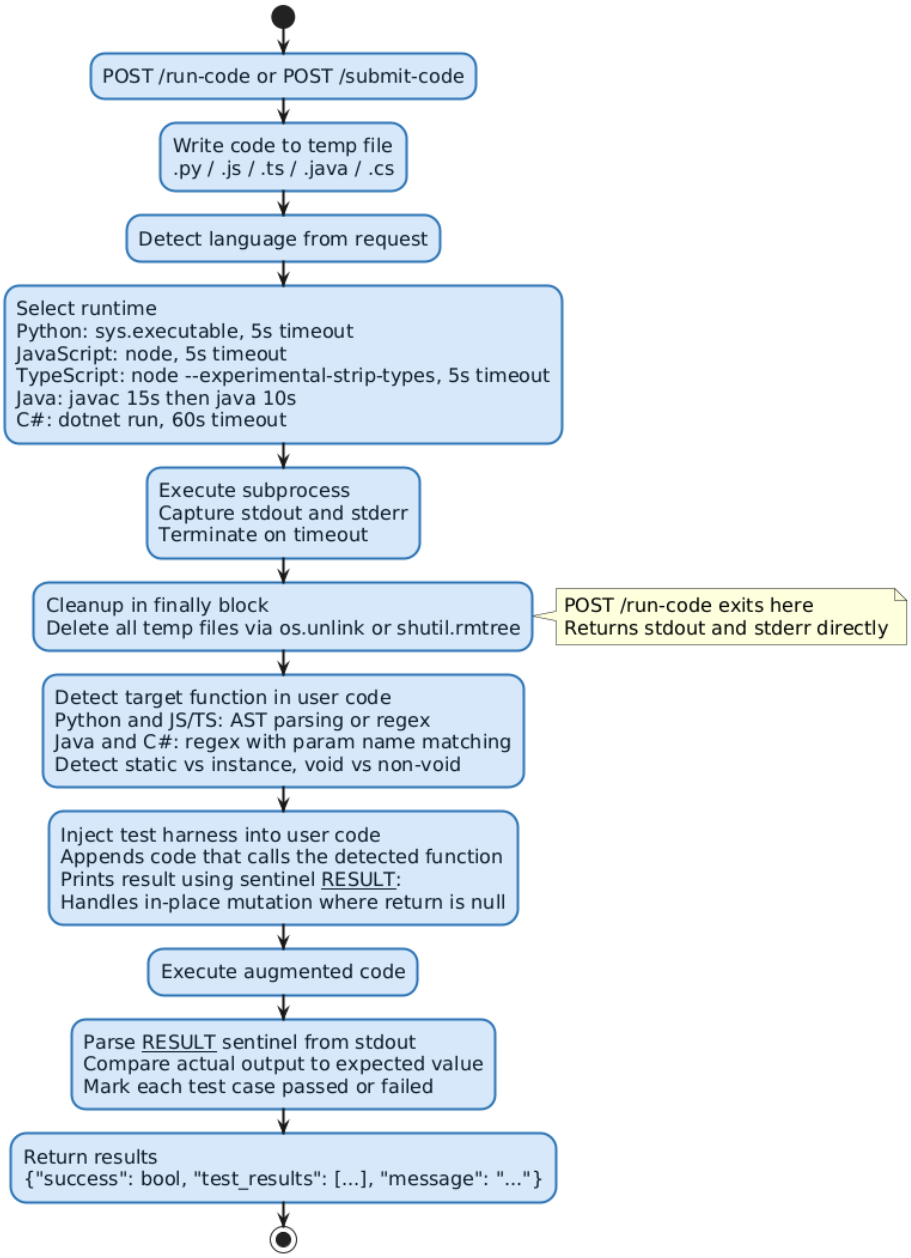

Figure 4.3: Flowchart of the code execution engine, tracing the path from an incoming request through runtime selection, subprocess execution, test harness injection, and result evaluation.

includes it in the `Authorization: Bearer <token>` header on all authenticated requests. Backend route handlers that require authentication call `verify_token()` from `core/auth.py`, which decodes and validates the JWT and returns the user ID for use in the handler.

Email verification is enforced at registration time. A secure random token generated by `secrets.token_urlsafe(32)` is stored on the user row together with a timestamp. A verification email is sent via SMTP (with TLS), and the user cannot log in until the link in that email is clicked. The verification endpoint only accepts tokens younger than twenty four hours, a rule enforced at the SQL level using `verification_sent_at > NOW() - INTERVAL '24 hours'`.

The experience point and levelling system is implemented in `services/users.py`. When a quiz result is submitted via `POST /save-quiz-result`, experience is awarded based on the number of correct answers. The threshold for levelling up follows the formula `xp_for_level = 100 * level`, so a level 1 user needs 100 experience points to reach level 2, a level 2 user needs 200 to reach level 3, and so on. When accumulated experience meets or exceeds the threshold, the level is incremented and the experience pool is reduced by the threshold amount, allowing carry over. A `while` loop handles the rare case where a single grant is large enough to cover multiple levels at once.

4.4 Database Design

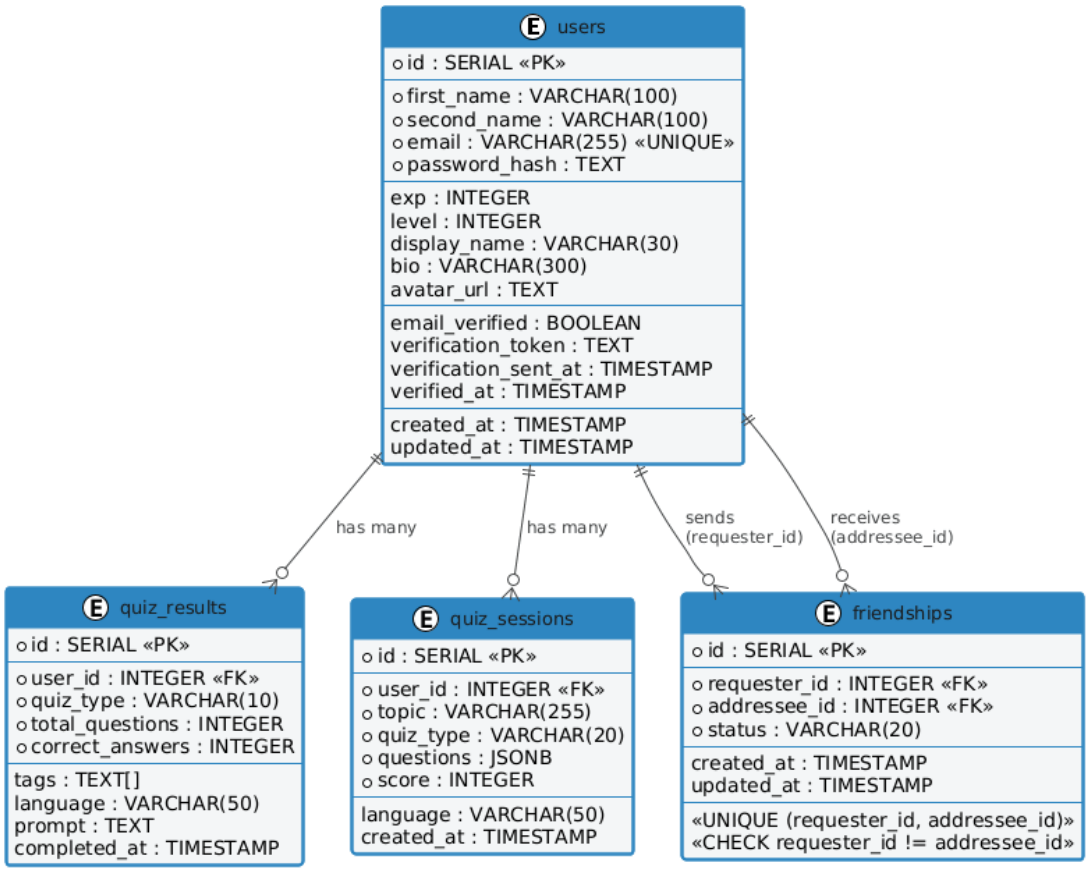

Figure 4.4: Entity relationship diagram for the four database tables. The users table is the central entity, with one to many relationships to both quiz_results and quiz_sessions. The friendships table models a directed self referential relationship on users, with a unique constraint preventing duplicate requests and a check constraint preventing self friendship.

As shown in Figure 4.4, the database schema consists of four tables: `users`, `quiz_results`, `friendships`, and `quiz_sessions`. The schema is created at application startup by `init_db()` using raw SQL `CREATE TABLE IF NOT EXISTS` statements, meaning the tables are created automatically on first run without requiring a separate migration tool.

4.4.1 Users Table

The `users` table is the central entity of the schema. In addition to authentication fields (`email`, `password_hash`), it stores profile data (`display_name`, `bio`, `avatar_url`), gamification state (`exp`, `level`), and email verification state (`email_verified`, `verification_token`, `verification_sent_at`, `verified_at`). The `email` column carries a `UNIQUE` constraint, enforcing that no two accounts share an email address at the database level, as a complement to the application level duplicate check.

4.4.2 Quiz Results Table

The `quiz_results` table records the outcome of completed quiz sessions. Notably, the full AI generated quiz content is not persisted in this table. It exists only in React component state for the duration of the session. Only the metadata necessary for the user's history and statistics is written to the database: the prompt that generated the quiz, the quiz type, the score, the predicted topic tags (stored as a PostgreSQL `TEXT[]` native array column), the programming language if applicable, and the completion timestamp. This design deliberately keeps the database lean and avoids the cost of serialising and storing potentially large JSON documents per session. Where a full record of the session is needed, the `quiz_sessions` table stores the complete question set as a `JSONB` column, and this supports the recent history view.

4.4.3 Friendships Table

The `friendships` table models the social graph as a directed edge list. Each row represents a friendship request from `requester_id` to `addressee_id` with a status of either `pending` or `accepted`. Two database level constraints prevent invalid states: a `UNIQUE(requester_id, addressee_id)` constraint prevents duplicate requests between the same pair of users, and a `CHECK (requester_id != addressee_id)` constraint prevents self friendship. A confirmed mutual friendship is represented by a single row with `status = 'accepted'`. The application layer treats both users as friends regardless of which direction the original request was sent.

4.5 Frontend Architecture

The frontend is a single page application built with React [6] and TypeScript [45], compiled by Vite [13]. At runtime the browser loads a single HTML document (`index.html`) containing script tags pointing to the Vite generated JavaScript

bundles. React Router handles client side navigation, mapping URL paths to page components without triggering full page reloads. Nginx's `try_files $uri /index.html` directive ensures that direct URL access or page refresh always returns the root HTML document so that React Router can take control.

4.5.1 Route Structure

The application exposes eleven routes, defined in `App.tsx`. Six of these routes are protected by an authentication guard that redirects unauthenticated users to the login page: `/prompt`, `/quiz`, `/code-sandbox`, `/profile`, `/social`, and `/stats`. The remaining five routes are publicly accessible: `/` (landing), `/login`, `/register`, `/about`, and `/user/:userId` (public profile). This separation reflects the application's requirement that content generation and social features are gated behind an account, while public profiles and informational pages are accessible without authentication.

4.5.2 State Management

Global application state is managed by a single hand rolled `AuthContext` built from React's `createContext` and `useState` primitives. The context exposes five values: `user` (the authenticated user object), `token` (the raw JWT string), `isAuthenticated`, `isLoading`, and two mutation functions `login` and `logout`. On login, both the user object and the token are written to `localStorage` so that the session survives a page refresh. On page load, `AuthContext` reads from `localStorage` to rehydrate the session without requiring a new login for the current browser tab.

All other state, quiz content, editor content, social list data, is local component state managed with `useState` hooks inside the relevant page component. This approach avoids the complexity of a global state management library for a codebase of this size, and the clear separation between global authentication state and local UI state makes the data flow straightforward to reason about.

4.5.3 Key Page Components

`PromptPage` is the primary entry point to quiz generation. It renders the `PromptForm` component, which collects the user's topic prompt, quiz type (MCQ or coding), programming language (for coding quizzes), and number of questions. On submission, it posts to `POST /prompt` and awaits the complete enriched quiz payload. On success, it navigates to either `/quiz` or `/code-sandbox`, passing the quiz data via React Router's location state.

`QuizPage` receives the MCQ quiz data from location state and renders the questions one at a time, tracking the user's selected answer and score in local state. Session state is also persisted to `sessionStorage` under a key derived from the user id and session id, so that a refresh of the page does not lose in progress answers.

`CodeSandboxPage` renders an interactive coding environment for the generated coding challenges, built on top of the Monaco Editor [46]. The editor, language selector, test case panel, and results panel are all managed as local state within this component. Submissions are sent to `POST /submit-code` and the per case pass or fail results are rendered inline next to the expected versus actual output.

`SocialPage` handles the full friends lifecycle: searching for users by display name, sending friend requests, accepting or rejecting incoming requests, and viewing the current friends list.

4.6 End to End Data Flow

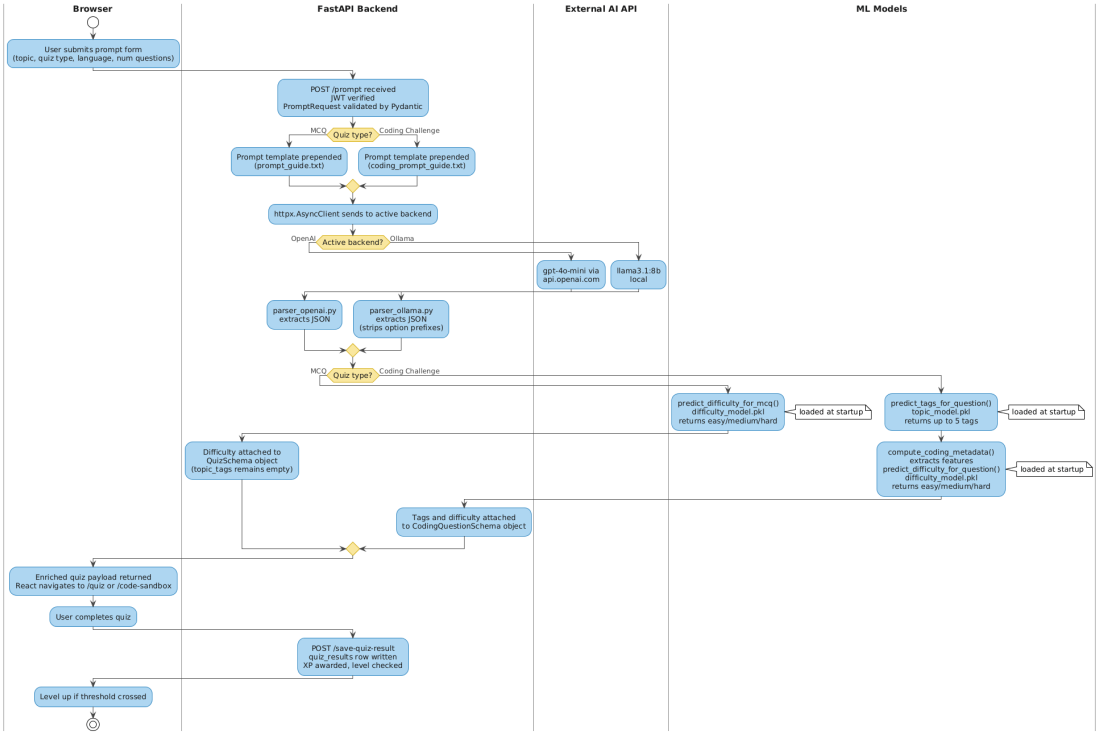

Figure 4.5: End to end data flow for a quiz generation session, showing the request path from browser through JWT authentication, AI generation via OpenAI or Ollama, and ML enrichment. MCQ questions receive a predicted difficulty label only, while coding challenges receive both difficulty and topic tags. Both models are loaded at startup as in process singletons. The session concludes with results persisted and experience points awarded.

To make the interaction between components concrete, this section traces the complete path of a quiz generation and submission session, as illustrated in Figure 4.5. The authenticated user submits a form at `/prompt` requesting five Python coding challenges on a chosen topic. The browser sends `POST /prompt` with the bearer JWT in the request header, and FastAPI validates the body against the `PromptRequest` Pydantic model before any handler logic runs. `ai_models.py` prepends the coding prompt template to the user's topic and dispatches the full prompt to the active backend via `httpx.AsyncClient` with a 120 second timeout. When the response arrives, the parser extracts the array of question objects and enriches each one: topic tags are obtained from `predict_tags_for_question()`

and a difficulty label from `predict_difficulty_for_question()`, both of which consult their in memory model singleton. The enriched `CodingQuestionSchema` list is serialised and returned, and the frontend stores it in local state before navigating to `/code-sandbox`. Submissions from the sandbox are sent to `POST /submit-code`, executed against the stored test cases by the code executor, and the pass or fail result list is rendered inline. On session completion, `POST /save-quiz-result` writes a `quiz_results` row and awards experience points, levelling up the user if the threshold is crossed.

4.7 Deployment Architecture

The system is deployed locally using Docker Compose on a single host. Containers communicate over Docker's default bridge network by service name (the backend resolves the database at hostname `db`, and Nginx proxies requests to `backend:8000`). The containerised architecture is inherently portable: the same `docker-compose.yml` could run on any Linux host provisioned on AWS EC2, Google Compute Engine, or DigitalOcean with no code changes. A production deployment would additionally require TLS termination (Nginx with Let's Encrypt), a process supervisor, and a managed database service to decouple database lifecycle from application containers.

4.8 Design Decisions and Standards Compliance

The machine learning inference pipeline follows the scikit learn `Pipeline` convention [26], which ensures that all preprocessing steps are encapsulated in a single serialisable object. This prevents training and serving skew: the same `TfidfVectorizer` vocabulary and IDF weights, and the same `StandardScaler` parameters that were fitted on the training data, are applied at inference time without any manual re implementation. Separating the training code from the inference code (the training scripts live in `ml_models/`, the inference wrappers live in `backend/ml/`) is a standard pattern for productionising machine learning models and reflects the principle that the artefact produced by training is a self contained, deployable unit. The REST API conforms to the OpenAPI 3.0 specification [52], authentication uses JWT (RFC 7519 [47]) with HMAC SHA256 signing, and password storage uses bcrypt [48], consistent with OWASP guidance [49].

Chapter 5

System Evaluation

5.1 Introduction

This chapter evaluates the system against the six objectives defined in the Introduction. It presents quantitative results for both machine learning components, discusses the behaviour of the end to end application pipeline, and reflects critically on the limitations of the approaches taken. Where results fall short of ideal, the reasons are examined honestly rather than being simplified or ignored, as understanding why a component under performs is as important as knowing that it does.

The evaluation is structured in three parts. Sections 5.2 and 5.3 cover the two classifiers in detail, presenting training curves, model comparisons, confusion matrices, and per class analysis. Section 5.4 evaluates the application level objectives: the generation pipeline, the code execution engine, and the user features. Section 5.5 then maps the findings back to each objective explicitly.

5.2 Objective 3: Difficulty Classifier

5.2.1 Dataset and Class Balance

The difficulty classifier was trained on problems extracted from IBM Project CodeNet [4], with difficulty labels derived from acceptance rate percentile binning as described in the System Design chapter. Because the labels were produced by dividing the sorted acceptance rate distribution into three equal terciles, the resulting class distribution is by construction approximately balanced, as shown in Figure 5.1. Each class contains approximately 1,280 problems, giving a total dataset of roughly 3,840 labelled problems. This balance is a direct consequence of the percentile binning strategy and is an important positive property: it means

that a naive majority class baseline would achieve only around 33 percent accuracy, and it removes the need for class weighting or oversampling techniques [37] during training.

Figure 5.1: Class distribution of the difficulty labels derived from acceptance rate percentile binning across the CodeNet dataset. The near equal counts across easy, medium, and hard confirm that the tercile binning strategy produced a balanced training set.

5.2.2 Model Comparison

Seven candidate models were evaluated using five fold cross validation [14] scored on F1 macro: XGBoost [33], HistGradientBoosting, Gradient Boosting [34], Logistic Regression, Linear SVM [35], Random Forest [32], and k Nearest Neighbours. Validation accuracy scores are shown in Figure 5.2.

Although F1 macro was used as the scoring criterion during RandomizedSearchCV to ensure balanced evaluation across the three difficulty classes, 5.2 reports validation accuracy for each model to give an intuitive sense of overall prediction quality. The ranking of models is consistent across both metrics.

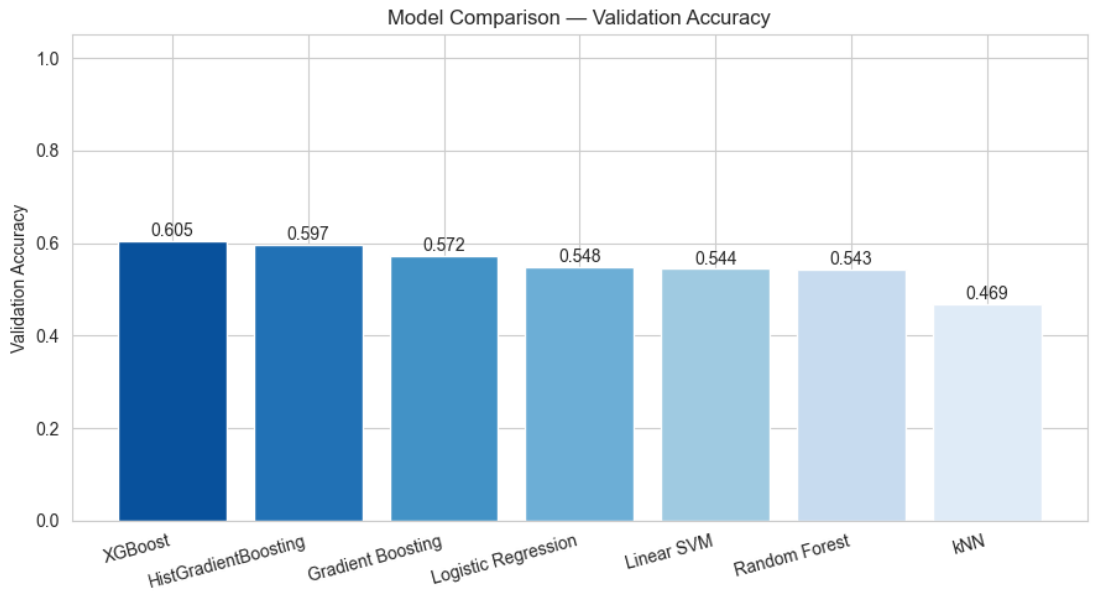

Figure 5.2: Validation accuracy comparison across all seven candidate difficulty classifier models after RandomizedSearchCV tuning. Model selection during cross validation was scored on F1 macro; accuracy is reported here for comparability. XGBoost achieved the highest validation accuracy at 0.605.

XGBoost [33] achieved the highest validation accuracy at 0.605, narrowly ahead of HistGradientBoosting at 0.597. Both ensemble methods substantially outperformed the weaker baselines of k Nearest Neighbours (0.469) and Linear SVM (0.544). The relatively small spread between the top four models (0.543 to 0.605) suggests that the difficulty classification task is intrinsically challenging, which is expected given that the labels are derived from a proxy signal rather than from ground truth human judgements [36].

5.2.3 Final Model Results

XGBoost was selected as the final model and evaluated on the held out test set. The confusion matrix is shown in Figure 5.3.

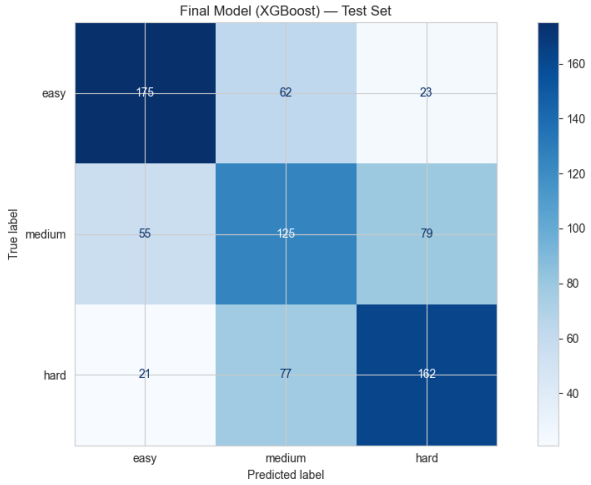

Figure 5.3: Confusion matrix for the final XGBoost difficulty classifier evaluated on the held out test set. Rows represent true labels and columns represent predicted labels.

The per class results derived from the confusion matrix are summarised in Table 5.1.

Class	Correct	Total	Recall	Misclassified as
Easy	175	260	0.67	Medium (62), Hard (23)
Medium	125	259	0.48	Easy (55), Hard (79)
Hard	162	260	0.62	Easy (21), Medium (77)
Overall	**462**	**779**		**Accuracy: 0.593**

Table 5.1: Per class performance of the XGBoost difficulty classifier on the test set, derived from the confusion matrix.

The model performs best on the easy class (recall 0.67) and worst on medium (recall 0.48). The medium class is consistently the hardest to classify across all models, which is a well known phenomenon in ordinal three class problems: the middle class shares boundary characteristics with both neighbours and therefore receives the most misclassifications in both directions. The hard class shows a notable asymmetry: 77 hard problems are misclassified as medium but only 21 as easy, which makes intuitive sense given that hard and medium problems are more similar to each other than hard and easy problems are.

5.2.4 Critical Analysis: The Proxy Label Problem

The most significant limitation of the difficulty classifier is one that cannot be resolved by changing the model: the labels themselves are a proxy. Acceptance rate reflects how often competitive programmers, ranging from beginners to experts, successfully solve a problem on a judging platform. This is a reasonable but imperfect signal for difficulty for several reasons. First, acceptance rate is influenced by factors beyond intrinsic problem difficulty, including the popularity of the problem (which determines how many attempts are made), the typical skill level of the platform's user base at the time the problem was active, and whether the problem has known edge cases that trip up otherwise strong solutions. Second, the percentile binning means that the label boundaries are arbitrary: a problem at the 33rd percentile is labelled easy and one at the 34th is labelled medium, even though their acceptance rates may differ by a fraction of a percent.

A test set accuracy of approximately 0.59 on a balanced three class problem represents a meaningful improvement over the random baseline of 0.33 and the majority class baseline of 0.33. However, it also means that roughly four in ten predictions are incorrect. In the context of the platform, this is acceptable because the difficulty label is presented to the user as a rough guide rather than a guarantee. A question labelled easy that is genuinely medium does not break the user experience. It merely provides slightly miscalibrated guidance. The value of the component lies in having some automated difficulty signal on AI generated content that would otherwise carry no label at all.

5.3 Objective 2: Topic Tag Classifier

5.3.1 Dataset Analysis

The topic tag classifier was trained on LeetCode [3] problems as a multi label classification task, since a single problem may be associated with multiple algorithmic topic tags. After removing tags with fewer than 60 samples, ten classes were retained: Array, String, Math, Hash Table, Database, Tree, Dynamic Programming, Linked List, Two Pointers, and Depth First Search. The class distribution before and after filtering is shown in Figure 5.4.

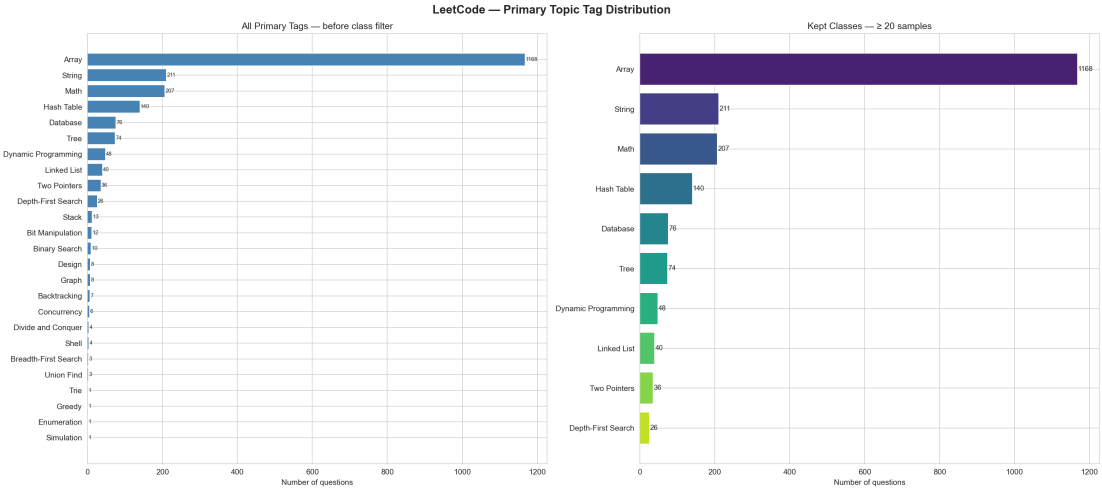

Figure 5.4: LeetCode topic tag distribution before filtering (left) and after applying the 60 sample minimum threshold (right). Array dominates with 1,168 examples. Tags such as Stack (13), Binary Search (10), and Graph (8) were removed due to insufficient samples.

The dataset is substantially imbalanced. Array accounts for 1,168 of the retained problems while Depth First Search contributes only 26. This imbalance is the dominant factor shaping classifier performance and is the root cause of the low F1 macro scores observed across all models [27]. A model can achieve high accuracy simply by performing well on Array and String, which together account for over 45 percent of the dataset, while performing poorly on the minority classes. This tension between accuracy and macro F1 is visible throughout the results.

Figure 5.5 shows the word count distribution per class. Database problems are notably longer on average, which is consistent with their SQL heavy descriptions. Tree and Linked List problems tend to be shorter.

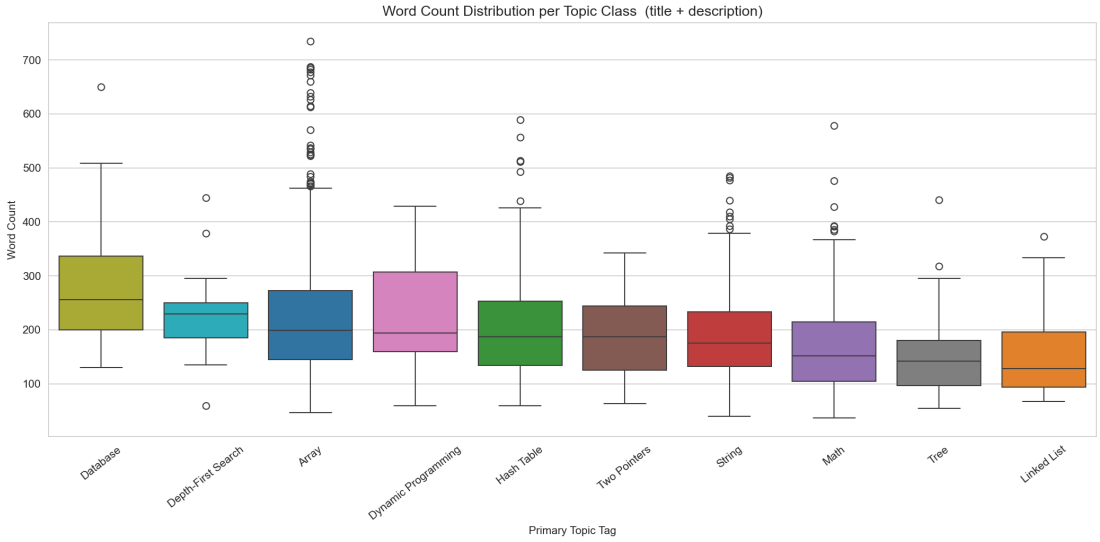

Figure 5.5: Word count distribution per topic class for the combined title and description text. Database problems are consistently longer than other classes. The wide variance within classes reflects the diverse nature of problems within each topic.

5.3.2 Feature Analysis

Figure 5.6 shows the top 30 terms across the entire corpus after English stop word removal. Terms such as `example`, `input`, `output`, and `nums` appear across all classes and provide little discriminative signal. The more informative terms are those that appear prominently in only one or two classes.

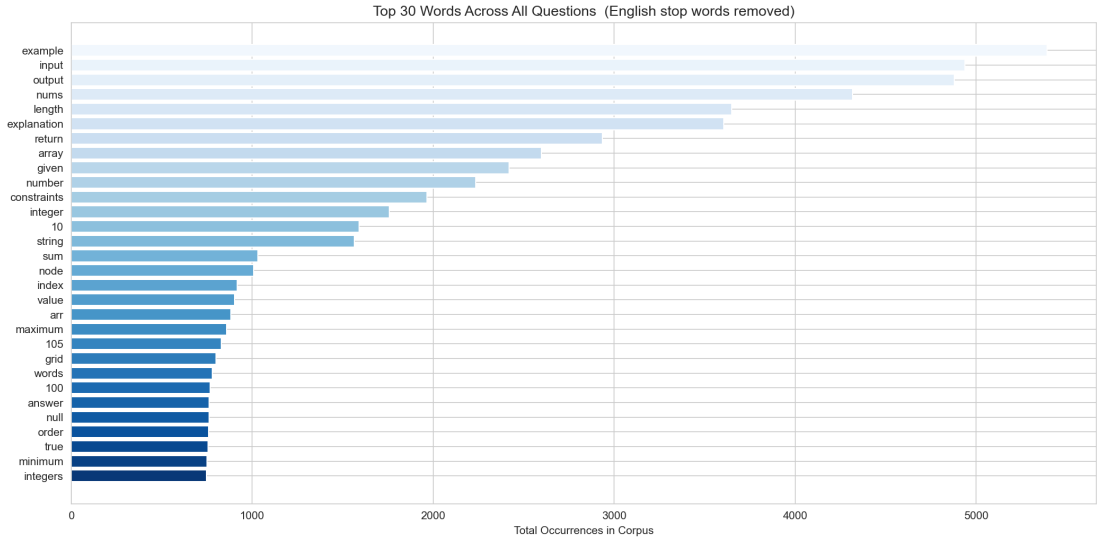

Figure 5.6: Top 30 terms across the full LeetCode corpus after English stop word removal. High frequency generic terms such as `example`, `input`, and `output` appear across all topic classes and provide limited discriminative value for classification.

The per class feature analysis from the final SGD Classifier is shown in Figure 5.7. This confirms that the model has learned genuinely meaningful signals: Database problems are characterised by terms such as `table`, `column`, and `column type`; Linked List problems by `linked`, `head`, and `nodes list`; Tree problems by `tree`, `root`, and `binary tree`. The features learned for Dynamic Programming are notably less clean, including general graph terms such as `graph` and `cities`, which explains the model's weaker performance on that class.

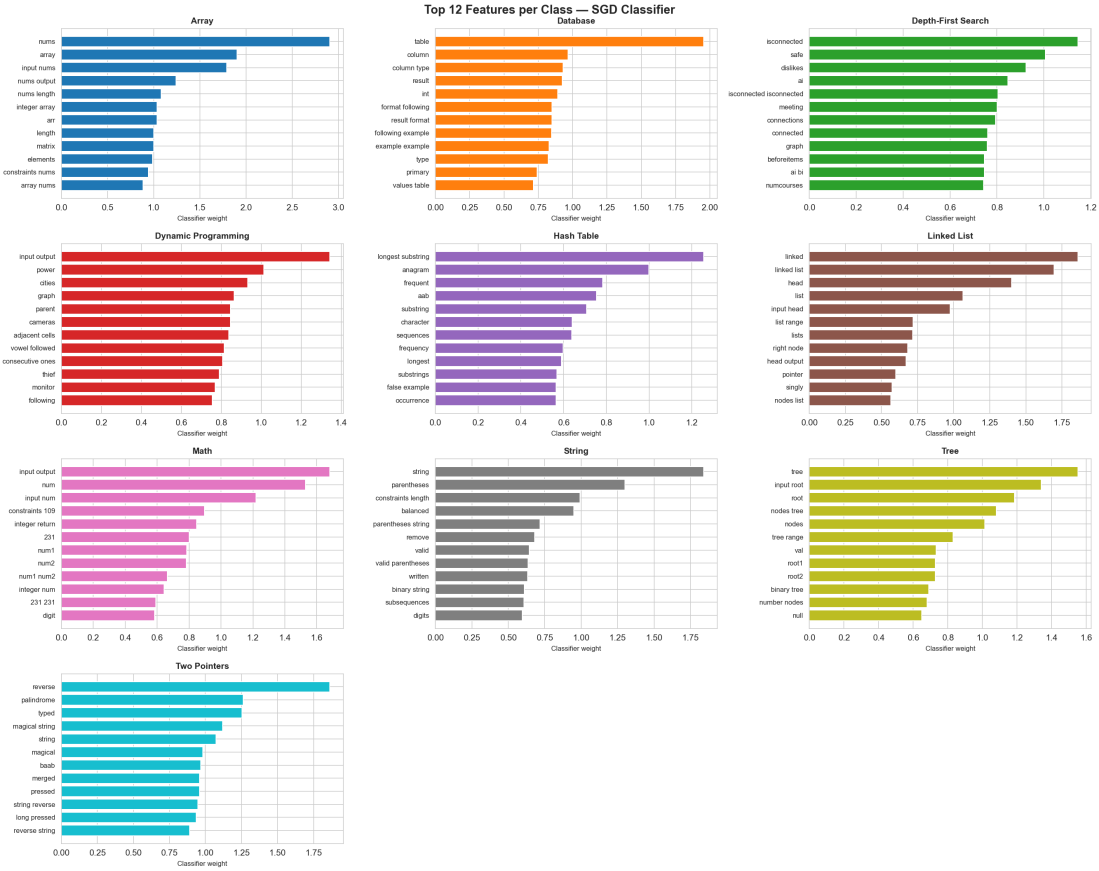

Figure 5.7: Top 12 classifier weight features per topic class for the final SGD Classifier. Classes with distinctive vocabulary (Database, Linked List, Tree) show clean, semantically meaningful features. Dynamic Programming and Hash Table show more overlap with other classes, which corresponds directly to their lower F1 scores.

5.3.3 Model Comparison

Seven candidate models were evaluated on the validation set. Model selection during training was performed using F1 micro, consistent with the multi label objective and the use of `GridSearchCV`. For evaluation and comparison, F1 macro is reported as the primary metric, as it provides a more informative view of performance across all classes in the presence of class imbalance [29]. The models correspond directly to the training configurations defined in the pipeline: logistic regression with word features, logistic regression with combined word and character features, an SGD classifier with word features, and a classifier chain of logistic regressions with

combined features. The results are shown in Figure 5.8.

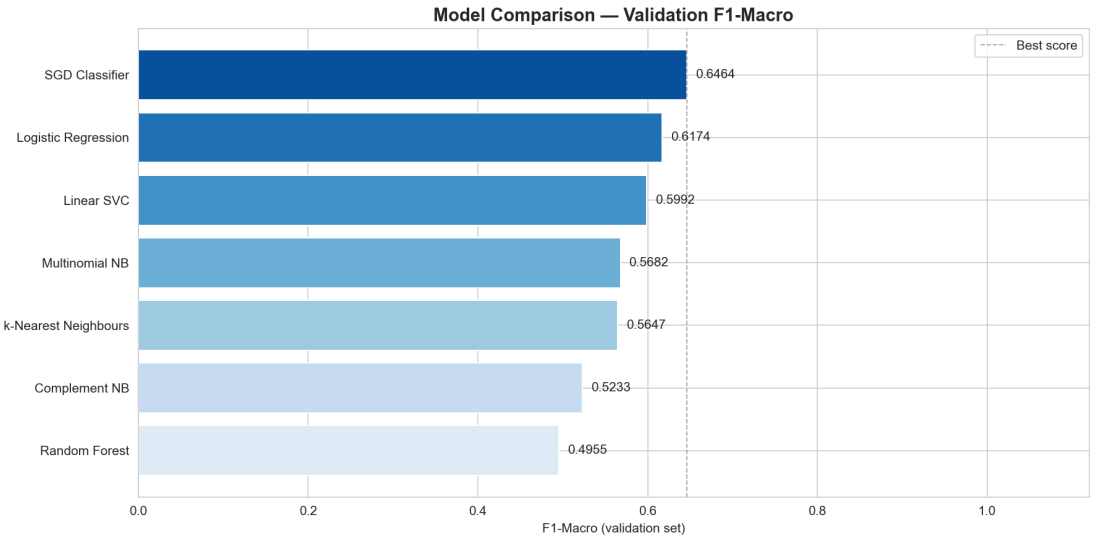

Figure 5.8: Validation set F1 macro comparison across the seven candidate topic tag classifier models. The SGD classifier achieved the highest F1 macro and was therefore selected as the final deployed model.

Figure 5.9 presents all three metrics (accuracy, F1 macro, and F1 weighted) for every model side by side, illustrating the gap between accuracy and macro F1 that results from the class imbalance.

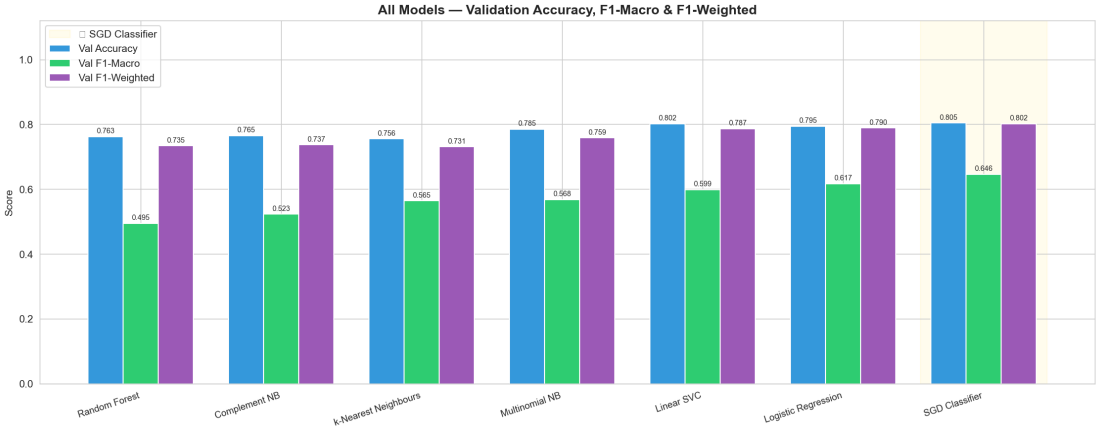

Figure 5.9: Validation accuracy, F1 macro, and F1 weighted for the seven candidate topic tag classifier models. The consistent gap between accuracy and F1 macro reflects the class imbalance in the LeetCode dataset, where high accuracy can be achieved by performing well on the dominant classes alone.

SGD Classifier achieved the highest validation F1 macro at 0.6464 and was selected as the deployed model. The effect of hyperparameter tuning via GridSearchCV is shown in Figure 5.10. For most models the gain from tuning was marginal, with the exception of Multinomial NB which improved substantially from 0.615 to 0.785 in validation accuracy after tuning, suggesting its default alpha parameter was poorly suited to this corpus.

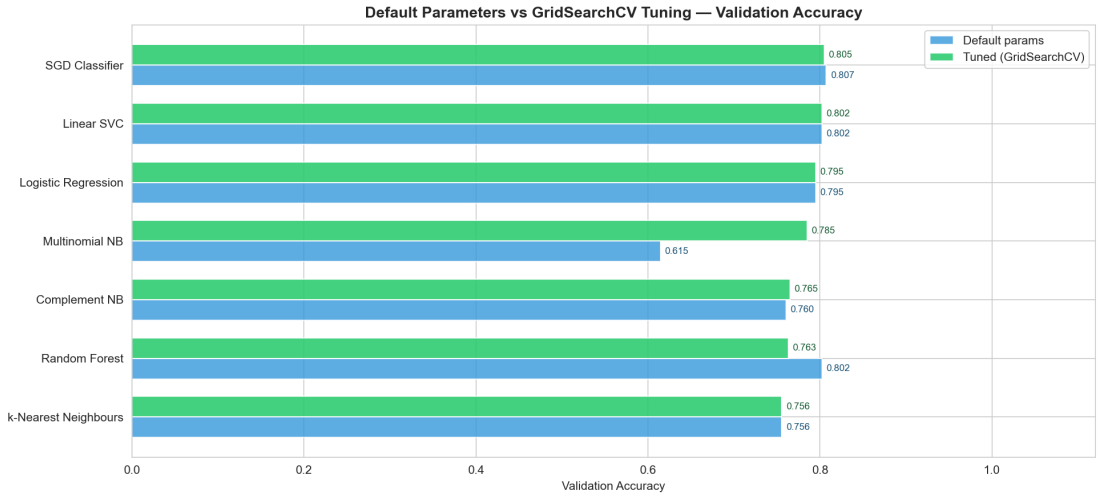

Figure 5.10: Comparison of default parameters versus GridSearchCV tuned parameters for each model on the validation set. The most notable improvement is Multinomial NB, which gained 17 percentage points in accuracy after tuning.

The five fold cross validation F1 macro score distributions are shown in Figure 5.11. SGD Classifier (0.588 ± 0.019) and Linear SVC (0.581 ± 0.006) show the most stable performance. Multinomial NB shows high variance (0.575 ± 0.041), indicating sensitivity to the particular fold split and making it a riskier choice for deployment.

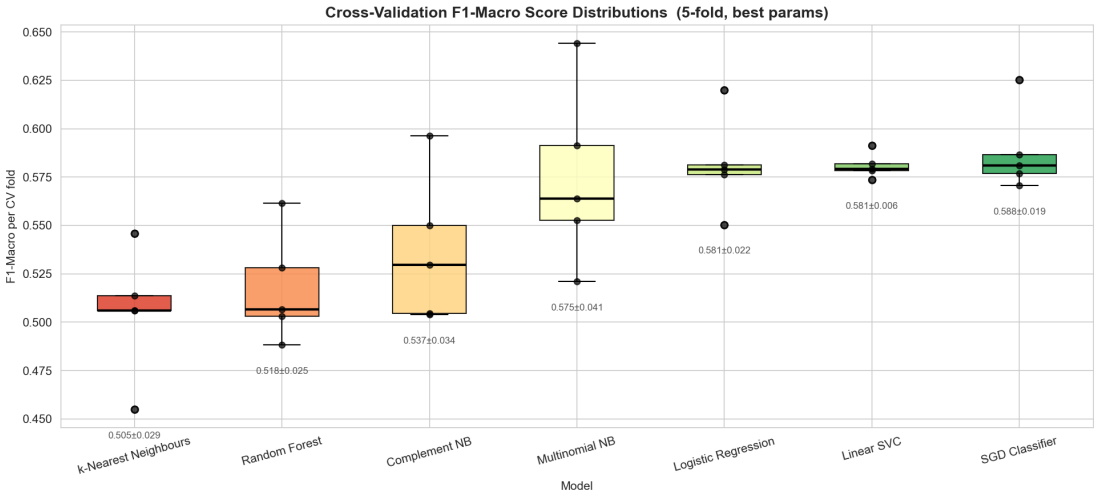

Figure 5.11: Cross validation F1 macro score distributions (five fold) for all models with best hyperparameters. SGD Classifier and Linear SVC show the tightest distributions, indicating the most consistent generalisation across folds.

5.3.4 Final Model Test Set Results

The SGD Classifier was evaluated on the held out test set. Final results are summarised in Table 5.2.

Metric	Validation	Test Set
Accuracy	0.805	0.805
F1 Macro	0.646	0.622
F1 Weighted	0.802	0.794

Table 5.2: Final SGD Classifier performance on validation and test sets. The 2.4 percentage point drop in F1 macro from validation to test set is modest and consistent with normal generalisation behaviour.

The per class F1 scores across all models on both validation and test sets are shown in Figures 5.12 and 5.13.

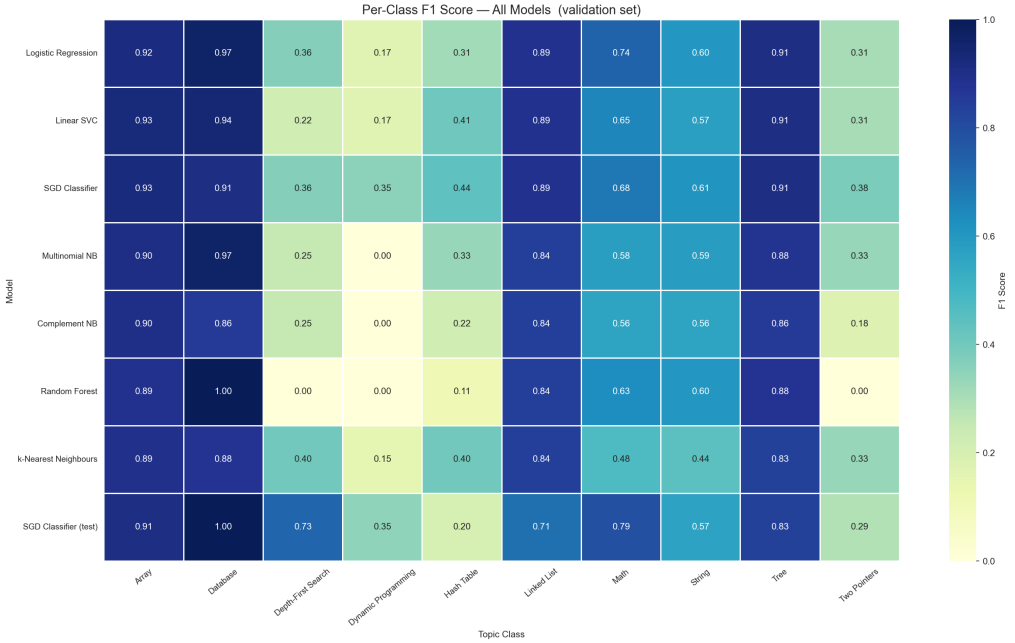

Figure 5.12: Per class F1 scores for all models on the validation set, with the SGD Classifier test set results shown in the final row. Array, Database, and Tree are consistently strong across all models. Dynamic Programming, Hash Table, and Two Pointers are consistently the weakest classes.

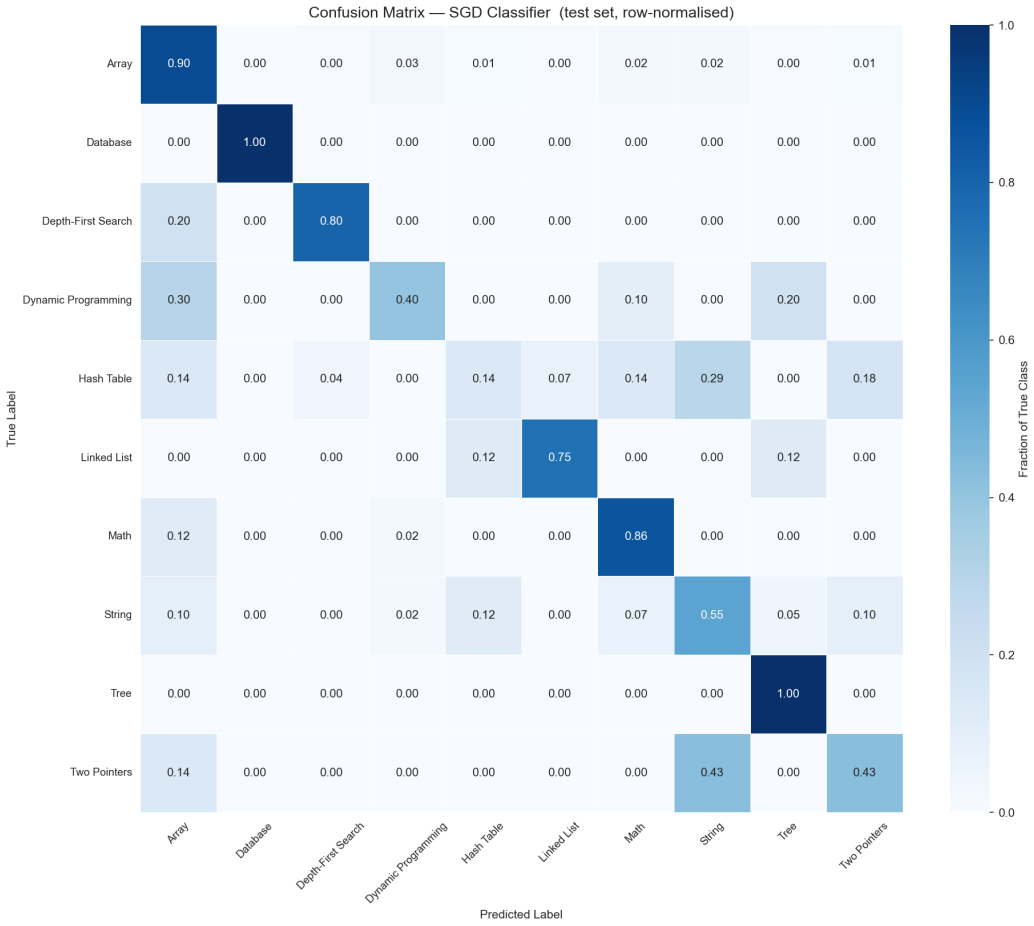

Figure 5.13: Row normalised confusion matrix for the SGD Classifier on the test set. Array (0.90), Database (1.00), Math (0.86), and Tree (1.00) show strong diagonal dominance. Dynamic Programming (0.40) and Two Pointers (0.43) show the highest off diagonal confusion, primarily misclassified as Array.

The per class results for the final deployed SGD Classifier are presented in Table 5.3.

Topic Class	Test F1	Primary Confusion
Array	0.91	
Database	1.00	
Tree	0.83	
Math	0.79	
Linked List	0.71	Hash Table
Depth First Search	0.73	Array
String	0.57	Array
Two Pointers	0.29	String or Math
Dynamic Programming	0.35	Array
Hash Table	0.20	Array or String

Table 5.3: Per class F1 scores for the final SGD Classifier on the test set, with the primary misclassification direction noted for weaker classes.

5.3.5 Critical Analysis: Class Imbalance and Conceptual Overlap

Two factors dominate the weaknesses of the topic tag classifier. The first is class imbalance: Array accounts for 1,168 training examples while Depth First Search has only 26. This means the classifier has seen roughly 45 times as many Array examples, and when a minority class problem contains any vocabulary that also appears in Array problems (which is almost always true for terms like `nums`, `input`, and `length`), the model is biased towards predicting Array.

The second factor is conceptual overlap between classes. Dynamic Programming and Depth First Search both involve graph like reasoning, and their problem descriptions frequently share terms. Hash Table problems often involve sliding window or two pointer counting patterns that resemble both Array and Two Pointers problems. These overlaps are not an artefact of the model's weakness. They reflect genuine ambiguity in the label definitions themselves, which is consistent with the broader observation by Zhang and Zhou [29] that label correlation is a defining feature of multi label problems.

Training time for all models was extremely fast, as shown in Figure 5.14, with all models completing GridSearchCV in under seven seconds. This confirms that the computational cost of maintaining and retraining the classifier is negligible.

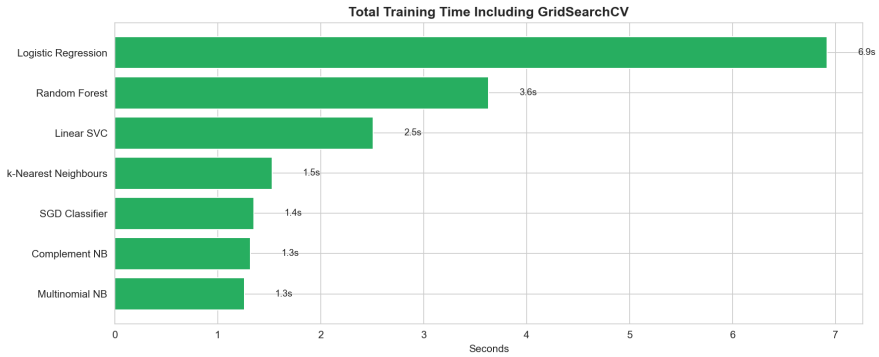

Figure 5.14: Total training time including GridSearchCV for each topic tag classifier model. All models complete training in under seven seconds, confirming that retraining is computationally trivial should the dataset be expanded in future.

Despite these limitations, a test set accuracy of 0.805 and F1 macro of 0.622 represents a genuinely useful classification capability for the platform's purpose. A user generating questions about Dynamic Programming may occasionally receive a question tagged as Array, but the majority of generated questions will receive a meaningful and correct topic label.

5.4 Application Level Evaluation

5.4.1 Objective 1: AI Generation Pipeline

The end to end quiz generation pipeline was tested manually across both AI backends and both quiz types. Both the OpenAI (`gpt-4o-mini`) and Ollama (`llama3.1:8b`) backends successfully produced well formed MCQ and coding challenge JSON in the vast majority of test runs. The OpenAI backend consistently produced clean, schema conforming JSON that parsed without intervention. The Ollama backend required additional post processing in `parser_ollama.py`, most notably stripping option prefix characters such as `"A: "` from MCQ option strings, which the OpenAI model did not include.

The most significant challenge encountered during development was JSON parsing reliability. LLMs do not guarantee valid JSON output, and both backends occasionally produced responses with minor formatting errors such as trailing commas, truncated option strings, or inconsistent field names when the prompt asked for a larger number of questions. This was partially mitigated by the strict schema specified in the prompt templates and by wrapping the parse step in try and except blocks that return meaningful error messages rather than crashing silently. This parsing fragility is a known limitation of prompt based LLM integration [20]

and is discussed further in the conclusion. The availability of structured output enforcement at the API level [22] offers a concrete path to resolving this limitation in a future iteration.

The 120 second `httpx` timeout was appropriate in practice. Requests to `gpt-4o-mini` for five questions typically completed in 5 to 15 seconds. Requests to the local Ollama backend were slower at 20 to 60 seconds depending on hardware, but remained within the timeout window.

5.4.2 Objective 5: Code Execution Engine

The code execution engine was tested manually across all five supported languages (Python, JavaScript, TypeScript, Java, C#) using both correct and intentionally incorrect solutions. The five second timeout correctly terminated infinite loop submissions in all tested cases. Python and JavaScript execution was reliable and fast. TypeScript required the transpile then execute step which added a small overhead but functioned correctly. Java and C# required careful handling of the class name and entry point expectations of their respective runtimes, and both worked reliably once these constraints were enforced in the execution logic.

Test case pass and fail evaluation was verified by submitting known correct solutions against the AI generated test cases and confirming that all cases passed, and then introducing deliberate off by one errors and confirming that the appropriate subset of test cases failed. The output comparison logic correctly handles trailing whitespace differences.

5.4.3 Objective 6: User Features

The authentication system, experience and levelling mechanic, quiz result persistence, and social friends network were all tested through manual interaction with the running application. JWT protected routes correctly rejected requests without a valid token. The levelling formula (`xp_for_level = 100 * level`) produced correct behaviour across level boundaries in manual testing.

The friends system was tested by creating two user accounts and exercising the full lifecycle: sending a friend request, accepting it, verifying mutual friendship, and then removing the friendship. The `UNIQUE(requester_id, addressee_id)` database constraint correctly prevented duplicate requests, and the `CHECK(requester\_id!=addressee\_id)` constraint correctly rejected self friendship attempts.

5.5 Evaluation Against Objectives

Table 5.4 summarises the outcome for each of the six objectives defined in the Introduction.

#	Objective	Status	Notes
1	Full-stack web application with AI powered MCQ and coding challenge generation supporting multiple AI backends	Met	Both OpenAI and Ollama backends functional. JSON parsing occasionally fragile on large requests.
2	Multi label topic tag classifier predicting algorithmic topic tags for generated questions	Met	Test accuracy 0.805, F1 macro 0.622. Class imbalance limits performance on minority classes.
3	Difficulty classifier predicting easy, medium, or hard in the absence of explicit human labels	Met	XGBoost test accuracy approx. 0.593. Proxy label ceiling acknowledged. Medium class hardest to classify.
4	Both classifiers integrated into the backend inference pipeline, enriching every generated question automatically	Met	Both models loaded as in process singletons. Inference adds negligible latency per question.
5	Code execution engine supporting multiple programming languages with test case evaluation	Met	All five languages functional. Five second timeout enforced. No kernel level sandboxing.
6	User accounts system with authentication, experience and levelling, and social friends network	Met	JWT auth, bcrypt passwords, full friends lifecycle, and experience system all functional.

Table 5.4: Summary of project objectives and their evaluation outcomes.

All six objectives were met. The primary areas of limitation are the difficulty classifier's reliance on a proxy label, the topic classifier's performance on minority

classes, and the JSON parsing fragility of the AI generation pipeline. These limitations are not failures of implementation but reflect genuine challenges in the problem domain, and each is discussed further in the Conclusion chapter.

Chapter 6

Conclusion

6.1 Summary of Context and Objectives

This project set out to address a genuine gap in the landscape of technical education tools: the absence of a platform capable of generating structured, enriched programming quizzes and coding challenges on demand, tailored to any topic a learner wishes to practise. Existing solutions either offer static question banks that grow stale, or expose raw AI generated content without any organisational structure or quality signal. The platform developed in this dissertation bridges that gap by combining the generative power of large language models [1, 19] with two custom trained machine learning classifiers that automatically enrich every generated question with a predicted topic tag and a predicted difficulty level.

Six objectives were defined at the outset of the project. These covered the full-stack web application and its AI generation pipeline, the two classifiers and their integration into the backend inference layer, the multi language code execution engine, and the user account system with its gamification and social features. All six objectives were pursued simultaneously across an iterative development process grounded in agile principles [10, 11], with a GitHub based workflow, continuous integration pipeline [12], and containerised Docker Compose infrastructure [5, 44] supporting the full development lifecycle. Each objective maps to a concrete subsystem in the delivered software, and each has been evaluated against measurable criteria in Chapter 5.

6.2 Summary of Findings

The system evaluation chapter assessed each objective against measurable criteria. The key findings are summarised below.

The AI generation pipeline functioned reliably across both supported backends.

The OpenAI `gpt-4o-mini` backend consistently produced clean, schema conforming JSON for both MCQ and coding challenge quiz types. The Ollama `llama3.1:8b` backend required additional post processing to handle formatting inconsistencies in its output but performed correctly once these were addressed. JSON parsing fragility when requesting larger numbers of questions was identified as the primary limitation of this component, and the structured output mode now offered by the OpenAI API [22] offers a clear path to resolving it.

The topic tag classifier achieved a test set accuracy of 0.805 and an F1 macro of 0.622 across ten algorithmic topic classes. Performance was strong on classes with distinctive vocabulary, with Database achieving a perfect F1 of 1.00 and Array achieving 0.91. Performance was weaker on conceptually overlapping classes such as Hash Table (0.20) and Dynamic Programming (0.35), where genuine ambiguity in the label definitions limits the ceiling of any text based classifier [29]. The shrinkage regularised per label threshold optimisation, informed by the approach of Lipton et al. [31], produced noticeable gains over the default 0.5 threshold on the minority classes.

The difficulty classifier achieved a test set accuracy of approximately 0.593 on a balanced three class problem. The XGBoost model [33] outperformed all six alternatives on validation accuracy. The medium difficulty class was the hardest to classify across all models, which is consistent with its position as the boundary class between easy and hard. The fundamental limitation of this component is the proxy label: acceptance rate is a reasonable but imperfect stand in for human assigned difficulty [36].

Both classifiers were successfully integrated into the backend inference pipeline as in process singletons, adding negligible latency to the question generation flow. Every generated question is automatically enriched with topic tags and a difficulty label before being returned to the frontend, delivering on the fourth objective directly.

The code execution engine supported all five target languages (Python, JavaScript, TypeScript, Java, and C#) with a five second timeout and per test case pass or fail evaluation. Execution was reliable across all languages, with Java and C# requiring careful handling of runtime entry point conventions.

The user accounts system, including JWT authentication [47], bcrypt password hashing [48], the experience and levelling mechanic, and the full social friends lifecycle, functioned correctly in all manual testing scenarios.

6.3 Unexpected Insights

Several insights emerged during the project that were not anticipated at the outset.

The most significant was the discovery of how strongly class imbalance dominates

multi class text classification performance. It was initially assumed that accuracy would be the primary indicator of classifier quality, and it was only through the process of computing F1 macro alongside accuracy that the disparity became apparent: a model scoring 0.805 accuracy was simultaneously scoring only 0.622 on F1 macro, because it was essentially ignoring minority classes. This led to the adoption of per label threshold tuning and the use of F1 macro as the primary model selection criterion, which meaningfully changed which model was chosen for deployment. The lesson that accuracy can conceal minority class failure is well documented in the literature [27], but encountering it in a concrete project rather than in the abstract made the point land far more forcefully.

A second unexpected insight came from the difficulty classifier's label engineering. The decision to use acceptance rate as a proxy for difficulty initially felt like a pragmatic workaround, but the process of investigating why the medium class was consistently the hardest to classify revealed something genuinely interesting about the nature of difficulty itself: the boundaries between easy, medium, and hard are not crisp in any dataset, human or otherwise. Problems cluster continuously along a difficulty spectrum, and any binning strategy introduces arbitrary boundaries. This is not unique to this project and reflects a broader challenge in any ordinal classification problem derived from continuous data.

A third observation, more practical in nature, was the extent to which prompt engineering turned out to be as important as model selection for the quality of the AI generation pipeline. Minor changes to the structure and specificity of the prompt templates had a larger effect on output consistency than switching between model backends. This echoes findings from the broader prompt engineering literature [20, 21], and it has concrete implications for how the project's templates should be versioned and tested going forward. The prompts are basically part of the product itself, so they should be handled like code, properly reviewed, version controlled, and tested to make sure they still give the expected results.

6.4 Opportunities for Future Investigation

Several directions for extending and improving the platform were identified during the course of the project.

The most impactful improvement to the topic tag classifier would be to expand the training dataset to include more examples of the minority classes, or to acquire a dataset with a more balanced distribution across tags. Oversampling techniques such as SMOTE [37] applied to the TF IDF feature space, or the use of class weighted loss functions more aggressive than the current `balanced` setting, could also be explored. The adoption of a pre trained language model such as a fine tuned BERT variant [17] for feature extraction, rather than a bag of words TF IDF

approach, would likely yield stronger performance on the conceptually overlapping classes where surface vocabulary is insufficient to discriminate between labels.

For the difficulty classifier, the most meaningful improvement would be the introduction of human verified difficulty labels, even for a small subset of problems, to serve as ground truth for evaluating and calibrating the proxy label approach. The current model could also be extended to incorporate solution level features such as the complexity of accepted solutions or the distribution of approaches used, which would provide a richer signal than problem description text alone. Replacing the three way classification with a regression onto a continuous difficulty score, and binning only at presentation time, would also remove the arbitrary boundary artefact described above.

The JSON parsing fragility of the AI generation pipeline would benefit from the adoption of structured output modes now available in the OpenAI API [22], which constrain the model's output to a specified JSON schema at the API level rather than relying on prompt instructions alone. This would eliminate the most common class of parsing errors encountered during testing. Similar features are emerging across other LLM providers and would allow the dual backend architecture to be retained while reducing parser complexity on both sides.

From an infrastructure perspective, deploying the application to a cloud provider using the existing Docker Compose configuration as a starting point, and adding HTTPS termination, a managed database, and horizontal scaling for the backend, would make the platform suitable for real user traffic. The continuous integration pipeline built during the project provides a strong foundation for automating this deployment process. The code execution engine would also need to be replaced with a properly sandboxed runner such as a dedicated judging system or an isolated container per submission before the platform could responsibly accept code from untrusted users.

A further direction, tangential but interesting, would be to feed the enriched annotations back into the generation prompt. Once the system knows that it has just produced an Array problem of medium difficulty, the next request could be steered explicitly towards a different topic or a harder level, producing a more balanced practice session without requiring the user to manage that breadth manually. This closes the loop between the generative and discriminative components of the system in a way that the current architecture hints at but does not yet exploit.

6.5 Closing Remarks

This project demonstrates that it is entirely feasible to build a full stack educational platform in which AI generated content is automatically enriched by custom trained machine learning models, all within the scope of a final year applied project. The

combination of large language model generation with lightweight, fast scikit learn classifiers [2] proves to be a practical and effective architecture: the LLM handles the creative and domain general task of producing coherent questions, while the classifiers handle the structured annotation task of labelling them.

The limitations identified throughout this dissertation, including the proxy label ceiling, class imbalance in the topic tagger, and JSON parsing edge cases, are not failures of ambition but markers of where the next iteration of this work begins. This project delivers a fully realised platform that integrates generation, tagging, grading, and execution of programming challenges with a social and gamified learning experience.

Bibliography

[1] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language models are few shot learners. In *Advances in Neural Information Processing Systems*, volume 33, pages 1877–1901, 2020.

[2] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Édouard Duchesnay. Scikit learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.

[3] Jaydeep Agravat. Leetcode dataset. `https://www.kaggle.com/datasets/jaydeepagravat94583/leetcode`, 2025.

[4] Ruchir Puri, David S. Kung, Geert Janssen, Wei Zhang, Giacomo Domeniconi, Vladimir Zolotov, Julian Dolby, Jie Chen, Mihir Choudhury, Lindsey Decker, Veronika Thost, Luca Buratti, Saurabh Pujar, Shyam Ramji, Ulrich Finkler, Susan Malaika, and Frederick Reiss. CodeNet: A large scale AI for code dataset for learning a diversity of coding tasks. In *NeurIPS Datasets and Benchmarks Track*, 2021.

[5] Dirk Merkel. Docker: Lightweight Linux containers for consistent development and deployment. *Linux Journal*, 2014.

[6] Meta Open Source. React: The library for web and native user interfaces. `https://github.com/facebook/react`, 2013. Accessed: Apr. 2026.

[7] F5 Inc. NGINX: High performance load balancer, web server, and reverse proxy. https://nginx.org, 2025. Accessed: Apr. 2026.

[8] Sebastián Ramírez. FastAPI. https://fastapi.tiangolo.com, 2025. Accessed: Apr. 2026.

[9] PostgreSQL Global Development Group. PostgreSQL: About. https://www.postgresql.org/about, 2025. Accessed: Apr. 2026.

[10] Kent Beck, Mike Beedle, Arie van Bennekum, Alistair Cockburn, Ward Cunningham, Martin Fowler, James Grenning, Jim Highsmith, Andrew Hunt, Ron Jeffries, Jon Kern, Brian Marick, Robert C. Martin, Steve Mellor, Ken Schwaber, Jeff Sutherland, and Dave Thomas. Manifesto for agile software development. https://agilemanifesto.org, 2001. Accessed: Apr. 2026.

[11] Kent Beck and Cynthia Andres. *Extreme Programming Explained: Embrace Change*. Addison Wesley, 2nd edition, 2004.

[12] GitHub Inc. GitHub Actions documentation. https://docs.github.com/en/actions, 2025. Accessed: Apr. 2026.

[13] Evan You. Vite: Next generation frontend tooling. https://vitejs.dev, 2025. Accessed: Apr. 2026.

[14] Ron Kohavi. A study of cross validation and bootstrap for accuracy estimation and model selection. In *Proceedings of the 14th International Joint Conference on Artificial Intelligence*, pages 1137–1143, 1995.

[15] James Bergstra and Yoshua Bengio. Random search for hyper parameter optimization. *Journal of Machine Learning Research*, 13:281–305, 2012.

[16] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, pages 5998–6008, 2017.

[17] Jacob Devlin, Ming Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics*, pages 4171–4186, 2019.

[18] Ollama. Ollama: Get up and running with large language models. https://ollama.com, 2025. Accessed: Apr. 2026.

[19] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. LLaMA: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.

[20] Pengfei Liu, Weizhe Yuan, Jinlan Fu, Zhengbao Jiang, Hiroaki Hayashi, and Graham Neubig. Pre-train, prompt, and predict: A systematic survey of prompting methods in natural language processing. *ACM Computing Surveys*, 55(9):1–35, 2023.

[21] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H. Chi, Quoc V. Le, and Denny Zhou. Chain of thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems*, volume 35, 2022.

[22] OpenAI. Introducing structured outputs in the API. `https://openai.com/index/introducing-structured-outputs-in-the-api`, 2024. Accessed: Apr. 2026.

[23] Karen Spärck Jones. A statistical interpretation of term specificity and its application in retrieval. *Journal of Documentation*, 28(1):11–21, 1972.

[24] Gerard Salton and Christopher Buckley. Term weighting approaches in automatic text retrieval. *Information Processing and Management*, 24(5):513–523, 1988.

[25] Juan Ramos. Using TF-IDF to determine word relevance in document queries. In *Proceedings of the First Instructional Conference on Machine Learning*, 2003.

[26] Lars Buitinck, Gilles Louppe, Mathieu Blondel, Fabian Pedregosa, Andreas Mueller, Olivier Grisel, Vlad Niculae, Peter Prettenhofer, Alexandre Gramfort, Jaques Grobler, Robert Layton, Jake Vanderplas, Arnaud Joly, Brian Holt, and Gaël Varoquaux. API design for machine learning software: Experiences from the scikit-learn project. In *ECML PKDD Workshop: Languages for Data Mining and Machine Learning*, pages 108–122, 2013.

[27] Nathalie Japkowicz and Shaju Stephen. The class imbalance problem: A systematic study. *Intelligent Data Analysis*, 6(5):429–449, 2002.

[28] Trevor Hastie, Robert Tibshirani, and Jerome Friedman. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. Springer, 2nd edition, 2009.

[29] Min Ling Zhang and Zhi Hua Zhou. A review on multi label learning algorithms. *IEEE Transactions on Knowledge and Data Engineering*, 26(8):1819–1837, 2014.

[30] Jesse Read, Bernhard Pfahringer, Geoff Holmes, and Eibe Frank. Classifier chains for multi label classification. *Machine Learning*, 85(3):333–359, 2011.

[31] Zachary C. Lipton, Charles Elkan, and Balakrishnan Narayanaswamy. Optimal thresholding of classifiers to maximize F1 measure. In *Joint European Conference on Machine Learning and Knowledge Discovery in Databases*, pages 225–239, 2014.

[32] Leo Breiman. Random forests. *Machine Learning*, 45(1):5–32, 2001.

[33] Tianqi Chen and Carlos Guestrin. XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 785–794. ACM, 2016.

[34] Jerome H. Friedman. Greedy function approximation: A gradient boosting machine. *The Annals of Statistics*, 29(5):1189–1232, 2001.

[35] Corinna Cortes and Vladimir Vapnik. Support vector networks. *Machine Learning*, 20(3):273–297, 1995.

[36] Zhi Hua Zhou. A brief introduction to weakly supervised learning. *National Science Review*, 5(1):44–53, 2018.

[37] Nitesh V. Chawla, Kevin W. Bowyer, Lawrence O. Hall, and W. Philip Kegelmeyer. SMOTE: Synthetic minority over sampling technique. *Journal of Artificial Intelligence Research*, 16:321–357, 2002.

[38] Tom Christie. Starlette: The little ASGI framework that shines. `https://www.starlette.io`, 2025. Accessed: Apr. 2026.

[39] Samuel Colvin. Pydantic: Data validation using Python type hints. `https://docs.pydantic.dev`, 2025. Accessed: Apr. 2026.

[40] Malhar Lathkar. *High Performance Web Apps with FastAPI: The Asynchronous Web Framework Based on Modern Python*. Apress, 2023.

[41] Robert C. Martin. *Clean Code: A Handbook of Agile Software Craftsmanship*. Prentice Hall, 2008.

[42] Michael Stonebraker and Lawrence A. Rowe. The design of POSTGRES. *ACM SIGMOD Record*, 15(2):340–355, 1986.

[43] TablePlus. Tableplus: Modern, native tool for database management. https://tableplus.com, 2025. Accessed: 2025.

[44] Docker Inc. Overview of Docker Compose. https://docs.docker.com/compose, 2025. Accessed: Apr. 2026.

[45] Microsoft. TypeScript: JavaScript with syntax for types. https://www.typescriptlang.org, 2025. Accessed: Apr. 2026.

[46] Microsoft. Monaco editor. https://microsoft.github.io/monaco-editor, 2025. Accessed: Apr. 2026.

[47] Michael Jones, John Bradley, and Nat Sakimura. JSON web token (JWT). IETF RFC 7519, 2015. https://www.rfc-editor.org/rfc/rfc7519.html.

[48] Niels Provos and David Mazières. A future adaptable password scheme. In *Proceedings of the USENIX Annual Technical Conference, FREENIX Track*, pages 81–91, 1999.

[49] OWASP Foundation. OWASP top ten. https://owasp.org/Top10, 2021. Accessed: Apr. 2026.

[50] Tim Bray. The JavaScript object notation (JSON) data interchange format. IETF RFC 7159, 2014. https://www.rfc-editor.org/rfc/rfc7159.html.

[51] Roy T. Fielding. Architectural styles and the design of network based software architectures. *PhD Thesis, University of California, Irvine*, 2000.

[52] OpenAPI Initiative. OpenAPI specification v3.0.3. https://spec.openapis.org/oas/v3.0.3, 2025. Accessed: Apr. 2026.

[53] Charles R. Harris, K. Jarrod Millman, Stéfan J. van der Walt, Ralf Gommers, Pauli Virtanen, David Cournapeau, et al. Array programming with NumPy. *Nature*, 585:357–362, 2020.

[54] Wes McKinney. Data structures for statistical computing in Python. In *Proceedings of the 9th Python in Science Conference*, pages 56–61, 2010.

[55] joblib developers. joblib: Running Python functions as pipeline jobs. https://joblib.readthedocs.io, 2025. Accessed: Apr. 2026.

[56] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016.

Appendix A

Installation and Repository

The full source code for this project is publicly available at:

`https://github.com/fionnmcgoldrick123/programming-quiz-app`

System Requirements

The platform is containerised and requires the following software to be installed on the host machine:

- Docker (version 24.0 or later)
- Docker Compose (version 2.20 or later)
- Git

Quick Start

1. **Clone the repository:**

```
git clone https://github.com/fionnmcgoldrick123/
   programming-quiz-app
cd programming-quiz-app
```

2. **Create the environment configuration file:**

```
cp .env.example .env
```

3. **Set the following required variables inside `.env`:**

- `OPENAI_API_KEY` — OpenAI API key for GPT-4o-mini generation
- `JWT_SECRET` — secret key for JWT signing (generate with `openssl rand -hex 32`)
- `POSTGRES_PASSWORD` — database password
- `SMTP_HOST`, `SMTP_USERNAME`, `SMTP_PASSWORD`, `SMTP_FROM` — email server credentials for user verification

4. **Build and start all services:**

```
docker-compose up --build
```

The build process takes approximately two to three minutes on first run. On completion, the services are accessible at the following addresses:

- Frontend: `http://localhost:3000`
- Backend API: `http://localhost:8000`
- API documentation: `http://localhost:8000/docs`

5. **To stop all services:**

```
docker-compose down
```

Retraining the Machine Learning Models

The serialised model artefacts are included in the repository and loaded automatically at startup. Should retraining be required, the difficulty classifier training script is located at `ml_models/difficulty_classifier/difficulty_predictor.py` and requires the `CODENET_PATH` environment variable to point to the root of the IBM Project CodeNet dataset. The topic tag classifier training script is located at `ml_models/tag_classifier/train.py` and reads from the included `leetcode.csv` file. Both scripts save their output to `difficulty_model.pkl` and `topic_model.pkl` respectively.